BigQuery for Data Warehousing Professionals

Welcome to the BigQuery for Data Warehousing Professionals module. Now that I've provided a high-level look at data warehousing architecture and the process of migrating, I'm going to take a deep dive into BigQuery itself.

Proprietary + Confidential

Learn how to...

Map Enterprise Data Warehouse concepts and components to BigQuery and Google data services

- Describe the key components of a successful data Warehouse implementation on BigQuery
- Identify best practices for implementing a data warehouse with BigQuery
- Use the BigQuery Web UI to access public datasets
- Perform queries using the BigQuery web UI and analyse query results using client libraries
- Combine ecommerce datasets to create enhanced datasets using BigQuery joins and unions

Google Cloud

Because BigQuery is the core of our data warehouse solution, it's important to know what it does and how to use it. The objective of this module is for you to be able to map Enterprise Data Warehouse concepts and components to BigQuery and Google data services.

Most data warehouses have a common set of components, so we'll examine how BigQuery supports these requirements. You'll be able to describe the key components of a successful data warehouse implementation on BigQuery.

There are many ways to use any product, so we'll describe some best practices and how they apply to your customer's data warehouse solution. You'll be able to identify best practices for implementing a data warehouse with BigQuery.

To facilitate training, testing, and benchmarking Google provides a wide range of public data sets. We'll look at some of these datasets and use them to demonstrate some of the concepts you learn.

The BigQuery UI will show you the results of queries you write, but ultimately customers want to visualize these results using reports and dashboards. You'll be able to perform queries using the BigQuery web UI and analyse query results using client libraries.

Because BigQuery supports flattened data without the traditional storage inefficiencies, I'll show you ways to write queries to join tables to facilitate

denormalizing. You’ll have the opportunity to combine ecommerce datasets to create enhanced datasets using BigQuery joins and unions.

Proprietary + Confidential

Agenda (1/3)

BigQuery Concepts

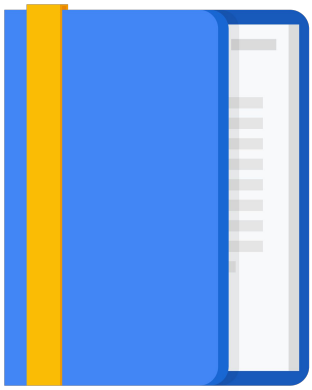
BigQuery Permissions and Security

Monitoring and Auditing

Schema Design

Partitioning and Clustering

Data Capture and Load Jobs



Google Cloud

This module covers a number of concepts. We'll start with some basic concepts of BigQuery. You need to know a bit about how it works if you're going to use it effectively.

Proprietary + Confidential

Agenda (2/3)

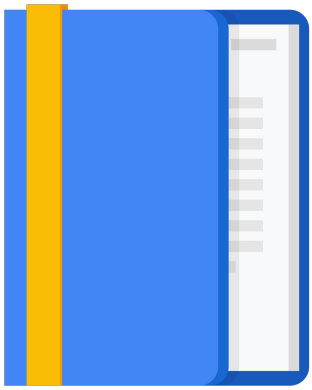
Handling Change and Slowly Changing Dimensions

Querying Data

Lab: Exploring Your Ecommerce Dataset with SQL in BigQuery

Managing Workloads and Concurrency

Analyzing Data



Proprietary + Confidential

Agenda (3/3)

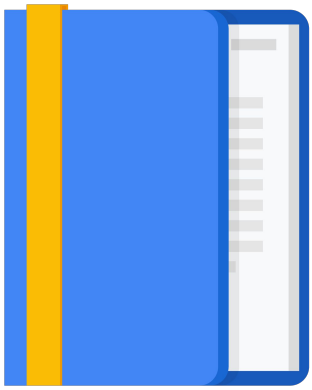
Sizing and Cost Management

Query Optimization

Storage Optimization

Lab: Creating a Data Warehouse Through Joins and Unions

Summary



Proprietary + Confidential

Common data warehouse concepts can be mapped to those in BigQuery

Data warehouse	Data mart	Data lake	Tables and views	Grants
BigQuery replaces a typical data warehouse hardware setup	BigQuery organizes data tables into units called datasets	BigQuery defines schemas and issues queries directly on external data sources	Function the same way as in a traditional data warehouse	Cloud IAM grants permission to perform specific actions

Google Cloud

In a traditional environment the data warehouse is comprised of hardware and software. Although BigQuery is serverless, you can think of it as replacing the on-premise hardware setup. It creates a repository for the consolidated set of operational and organizational data.

Data warehouses may be comprised of data marts, which are subsets of data typically aligned with a business line or team. In BigQuery, data marts can be implemented using datasets.

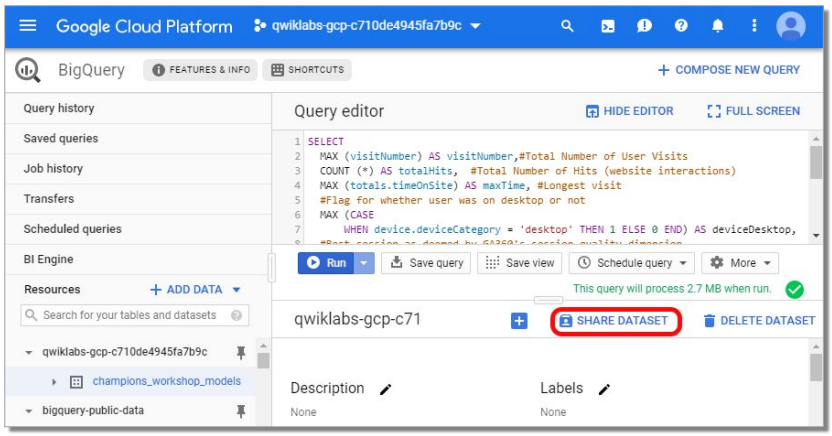
A data lake might contain files in Cloud Storage or Google Drive or transactional data in Cloud Bigtable. BigQuery can define a schema and issue queries directly on external data as federated data sources.

Database tables and views function the same way in BigQuery as they do in a traditional data warehouse, allowing BigQuery to support queries written in a standard SQL dialect which is ANSI 2011 compliant.

Cloud Identity and Access Management is used to grant permission to perform specific actions in BigQuery. This replaces the SQL GRANT and REVOKE statements that are used to manage access permissions in traditional SQL databases. Using the same security method for all services in Google Cloud means there's no requirement for a separate BigQuery administrator to manage security.

Proprietary + Confidential

Collaboration and fully-managed services can improve analysts' productivity



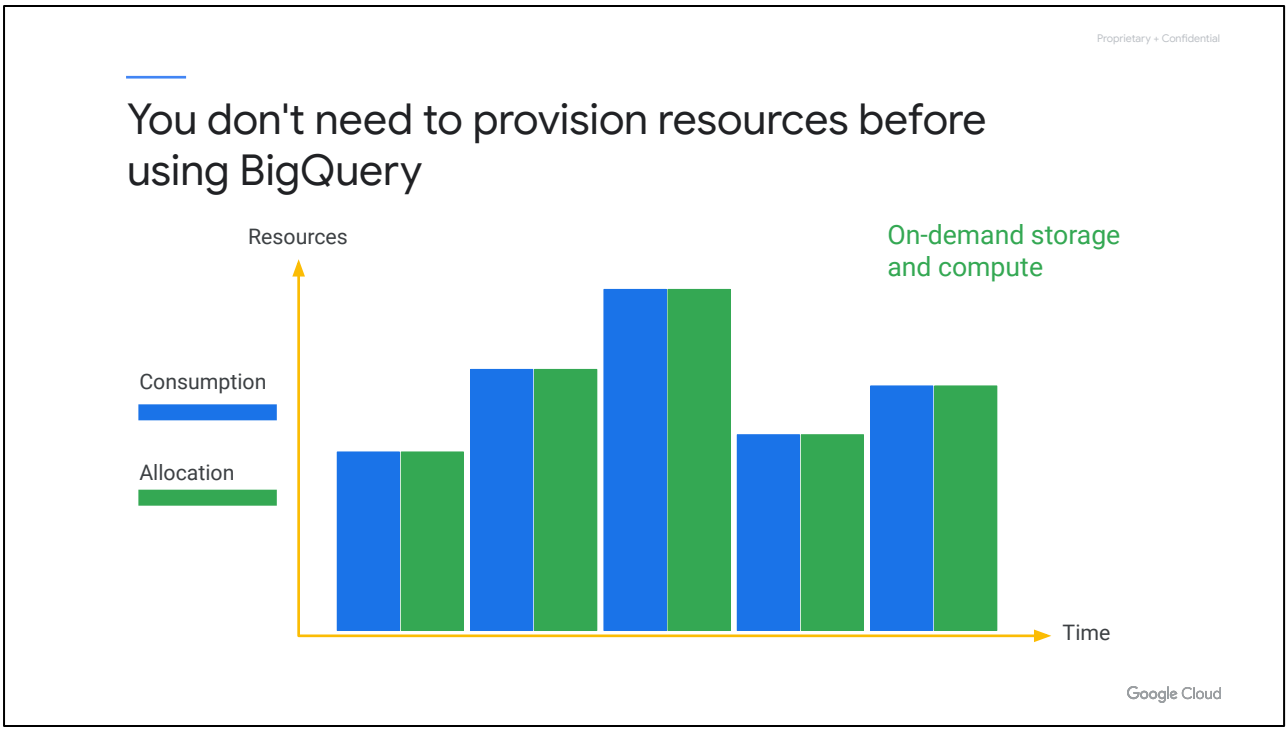
Google Cloud

Traditionally, onboarding new data analysts involved significant lead time. To enable analysts to run simple queries, you had to show them where data sources resided, configure ODBC connections and tools, and give them access rights. Using Google Cloud, you can greatly accelerate an analyst's time to productivity.

To onboard an analyst on Google Cloud, simply grant access to the relevant projects or datasets, introduce them to the Google Cloud console and BigQuery web UI, and share some queries to help them get acquainted with the data.

The Google Cloud console provides a centralized view of all assets in your Google Cloud environment. The most relevant asset to data analysts might be Cloud Storage buckets, where they can collaborate on files.

The BigQuery web UI presents the list of datasets that the analyst has access to. Analysts can perform tasks in the Google Cloud console according to the role you grant them, such as viewing metadata, previewing data, executing, and saving and sharing queries.



You don't need to provision resources before using BigQuery, unlike many RDBMS systems. BigQuery allocates storage and query resources dynamically based on your usage patterns.

Storage resources are allocated as you consume them and deallocated as you remove data or drop tables.

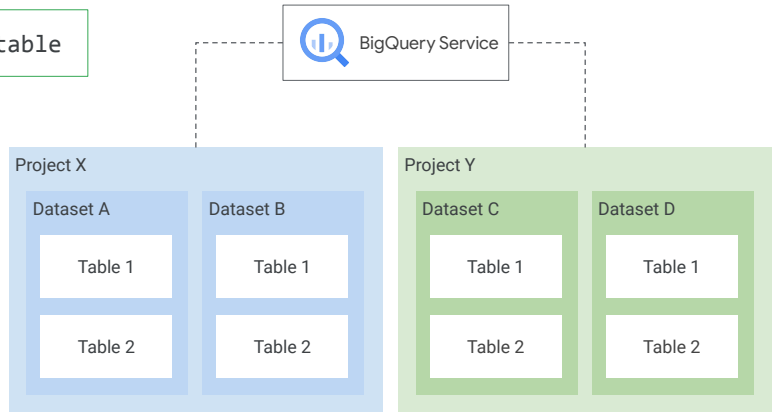
Query resources are allocated according to query type and complexity. Each query uses some number of slots, which are units of computation that comprise a certain amount of CPU and RAM.

You don't have to make a minimum usage commitment to use BigQuery. The service allocates and charges for resources based on your actual usage. By default, all BigQuery customers have access to 2,000 slots for query operations. You can also reserve a fixed number of slots for your project.

Proprietary + Confidential

BigQuery organizes data tables into units called datasets

`project.dataset.table`



What are some reasons to structure your information into:

- Datasets?
- Projects?
- Tables?

Google Cloud

BigQuery organizes data tables into units called datasets. These datasets are scoped to your Google Cloud project. When you reference a table from the command line, in SQL queries, or in code, you refer to it by using the construct: project.dataset.table.

These multiple scopes—project, dataset, and table—can help you structure your information logically. You can use multiple datasets to separate tables pertaining to different analytical domains, and you can use project-level scoping to isolate datasets from each other according to your business needs.

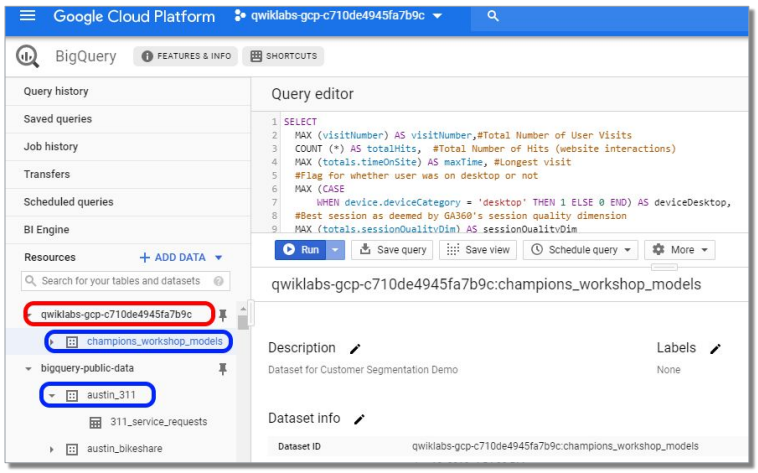
What are some reasons to structure your information into datasets, projects, or tables?

Well, reasons could include billing for projects, data mart/different permissions for datasets, primarily around flattened versus join for tables. We'll cover these in more detail.

Proprietary + Confidential

Organize datasets to manage workflows

Only datasets to which analyst has access



Google Cloud

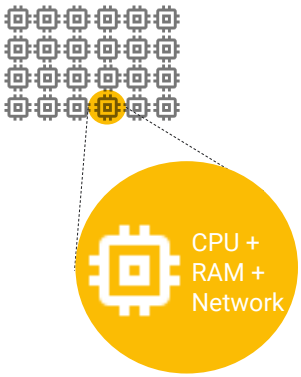
Because security can be set at either the project or dataset level, you can segment datasets into separate projects based on class of data or business unit, or consolidate them into common projects for simplicity.

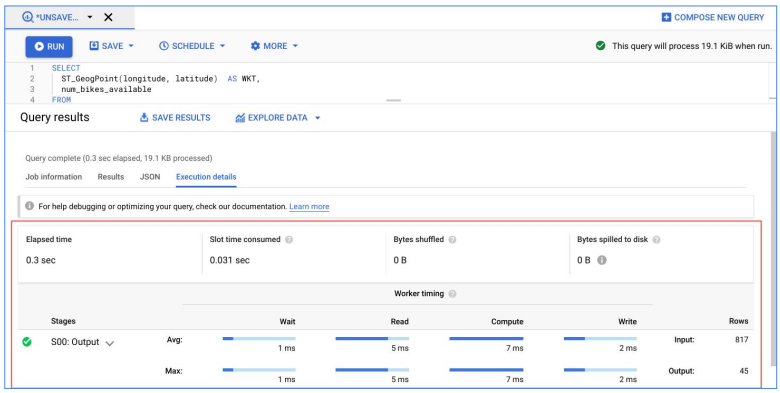
You can invite a data analyst to collaborate on an existing dataset in any limited role that you define. When a data analyst logs into the BigQuery web UI, they see only the datasets that have been shared with them across projects. The activities that they can perform against datasets varies, based on their role against each dataset.

Proprietary + Confidential

A BigQuery slot is a combination of CPU, memory, and networking resources

BigQuery slots





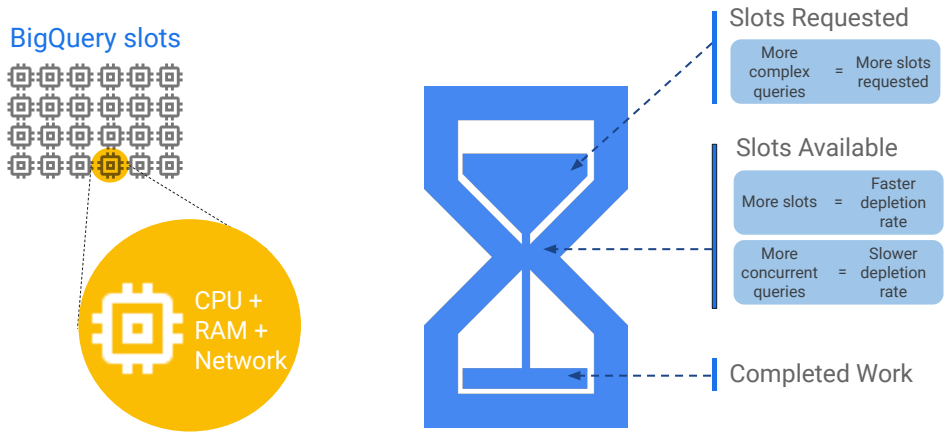
Google Cloud

BigQuery is implemented using a microservice architecture, so there are no virtual machines to configure and maintain. Under the hood, analytics throughput is measured in BigQuery slots. A BigQuery slot is a unit of computational capacity required to execute SQL queries. BigQuery automatically calculates how many slots are required at each stage in a query, depending on size and complexity.

A BigQuery Slot is a combination of CPU, memory, and networking resources. It also includes a number of supporting technologies and sub-services. Note that each slot doesn't necessarily have the same specification during query execution. Some slots may have more memory than others, or more CPU or more I/O.

Proprietary + Confidential

The actual number of slots allotted to a query depends on query complexity and project quota



Google Cloud

By default, each account has a quota limit of 2000 BigQuery slots for on-demand querying. A flat rate pricing model is available that provides reserved slots for customers who want more predictable pricing.

If you have reserved 10,000 slots but you have 30 concurrent queries that together ask for 15,000 slots, the queries will not get all the slots they ask for. That doesn't mean the queries won't run, they may just take longer.

Proprietary + Confidential

BigQuery stores repeated, nested fields in a columnar format

```
DocId: 10                    r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

```
DocId: 20                    r2
Links
  Backward: 10
  Backward: 30
  Forward:  80
Name
  Url: 'http://C'
```

DocID

value	r	d
10	0	0
20	0	0

Name.Url

value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward

value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward

value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

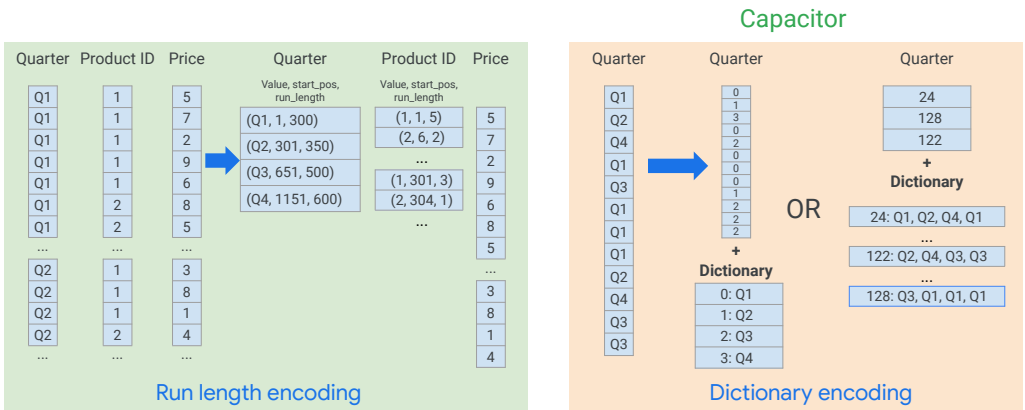
Dremel

Google Cloud

Internally, BigQuery uses columnar storage that supports semistructured data — nested and repeated fields. When Google published the Dremel paper in 2010, it explained how this structure is preserved within column store. Every column, in addition to its value, also stores two numbers — definition and repetition levels. This encoding ensures that the full or partial structure of the record can be reconstructed by reading only requested columns, and never requires reading parent columns, which is the case with alternative encodings.

Proprietary + Confidential

The data are also run-length encoded and dictionary-encoded



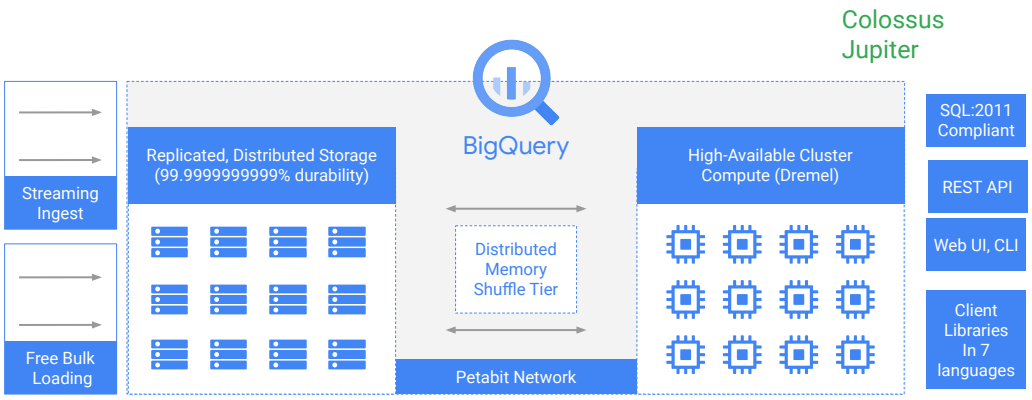
Examples taken from VLDB 2009 tutorial on Column Oriented Database Systems

Google Cloud

Columnar data is stored in a proprietary columnar format called Capacitor, which has a number of benefits for data warehouse workloads. BigQuery uses a proprietary format because it can evolve in tandem with the query engine, which takes advantage of deep knowledge of the data layout to optimize query execution. BigQuery uses query access patterns to determine the optimal number of physical shards and how they're encoded. Don't worry, you don't have to do anything – BigQuery does this for you automatically!

Proprietary + Confidential

The data is physically stored in a redundant way separate from the compute cluster



Google Cloud

BigQuery is implemented in two parts: a storage engine and an analytic engine as illustrated. The separation of compute and storage is a common theme in Google Cloud and works effectively because of Google's petabit network called Jupiter. Jupiter allows blazing fast communication between compute and storage.

BigQuery data is physically stored on Google's distributed file system, called Colossus, which ensures durability by using erasure encoding to store redundant chunks of the data on multiple physical disks. Moreover, the data is replicated to multiple data centers.

Proprietary + Confidential

BigQuery is a fully-managed service

- X Data aging
- X Storage management
- X Fault recovery
- X Query engine optimization
- X Hardware
- X Updates

Free up real people-hours by not having to worry about common tasks.

Google Cloud

BigQuery is a fully-managed service, which means that the BigQuery engineering team takes care of traditional administrative tasks. There's no hardware to manage and upgrades shouldn't require downtime or hinder system performance. Using managed services frees up people-hours by not having to worry about common administrative tasks.

Many traditional systems require resource-intensive processes to run at various intervals to reshuffle and sort data blocks and recover space. BigQuery has no equivalent of the process, because the storage engine continuously manages and optimizes how data is stored and replicated. Also, because BigQuery doesn't use indexes on tables, you don't need to rebuild indexes.

Proprietary + Confidential

BigQuery addresses backup and disaster recovery at the service level

```
CREATE OR REPLACE TABLE ch10eu.restored_cycle_stations AS
SELECT
*
FROM `bigquery-public-data`.london_bicycles.cycle_stations
FOR SYSTEM_TIME AS OF
TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 24 HOUR)
```

Query a point-in-time snapshot (up to 7 days)
Use it to create a backup

```
NOW=$(date +%s)
SNAPSHOT=$(echo "($NOW - 120)*1000" | bc)
bq --location=EU cp \
ch10eu.restored_cycle_stations@$SNAPSHOT \
ch10eu.restored_table
```

Restore a 2-min old copy (Deleted tables are flushed after 2 days or if recreated with same name)

Google Cloud

By maintaining a complete 7-day history of changes against your tables, BigQuery allows you to query a point-in-time snapshot of your data. You can easily revert changes without having to request a recovery from backups.

When a table is explicitly deleted, its history is flushed after 2 days.

You can recover a deleted table only if another table with the same ID isn't created in the same dataset after deletion. This is why you should be careful about using CREATE OR REPLACE TABLE – if you make a mistake with the table name it makes the table unrecoverable.

Proprietary + Confidential

Agenda

BigQuery Concepts

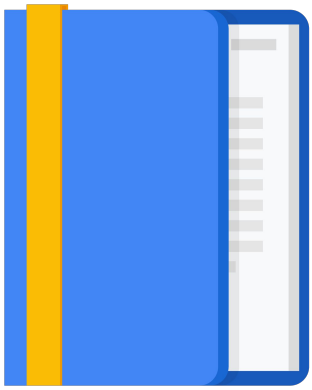
BigQuery Permissions and Security

Monitoring and Auditing

Schema Design

Partitioning and Clustering

Data Capture and Load Jobs

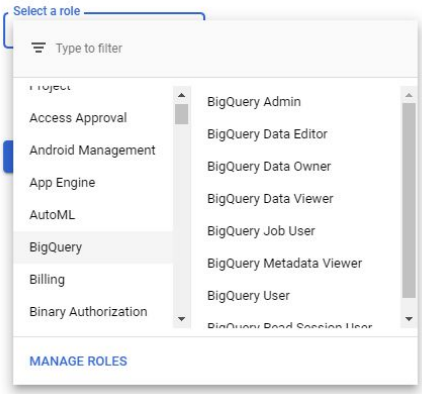


Google Cloud

Because data is the core of the data warehouse system, it‘s important to protect it from unauthorized users while allowing approved users access to the data they are allowed to use. In this lesson, we’ll discuss BigQuery permissions and security.

Proprietary + Confidential

BigQuery provides predefined roles for controlling access to resources



Grants

Cloud IAM grants permission to perform specific actions

IAM applies to datasets

Use authorized views to restrict at a finer-grained level

Google Cloud

In a traditional RDBMS system, you grant permissions to view or modify tables by creating SQL grants and applying them to a given user within the database system. In addition, some RDBMS systems allow you to grant permissions to users or groups in an external directory, such as LDAP. The BigQuery model for managing users and permissions resembles the latter model.

Google Cloud provides predefined roles for controlling access to all resources in Google Cloud. For BigQuery, there are predefined roles that can be assigned to users or groups. You can also create custom Cloud IAM roles consisting of your defined set of permissions, and then assign those roles to users or groups. You can assign a role to a Google email address or to a Google Workspace Group.

Using the same security model for all resources in Google Cloud rather than a separate model for BigQuery simplifies administration.

An important aspect of operating a data warehouse is allowing shared but controlled access against the same data to different groups of users. For example, finance, HR, and marketing departments all access the same tables, but their levels of access differ. Traditional data warehousing tools make this possible by enforcing row-level security. You can achieve the same results in BigQuery by defining authorized views, typically including the SESSION_USER() in the WHERE clause.

Proprietary + Confidential

Agenda

BigQuery Concepts

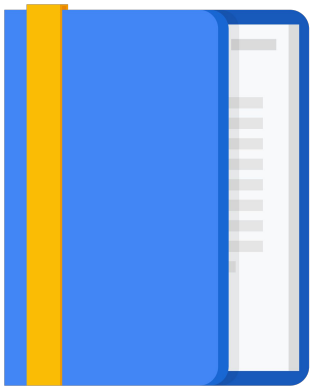
BigQuery Permissions and Security

Monitoring and Auditing

Schema Design

Partitioning and Clustering

Data Capture and Load Jobs

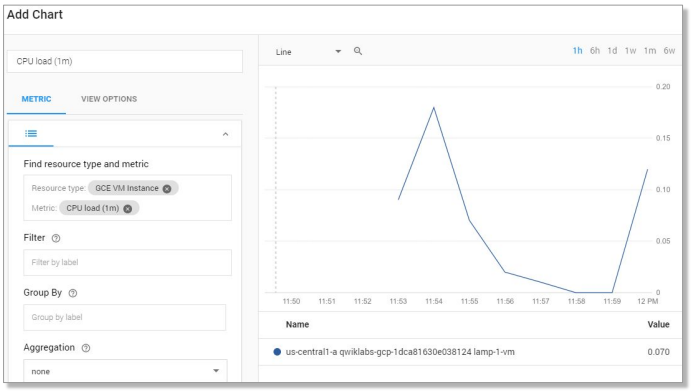


Google Cloud

Even though BigQuery is a managed service, it's often necessary to monitor usage and debug problems. In this lesson, we'll discuss monitoring and auditing as well as debugging common error messages.

Proprietary + Confidential

Monitor BigQuery using Cloud Monitoring



- Project/dataset metrics
- Queries in flight
- Query time
- Slot utilization (available/allocated)
- Tables
- Bytes stored
- Rows uploaded

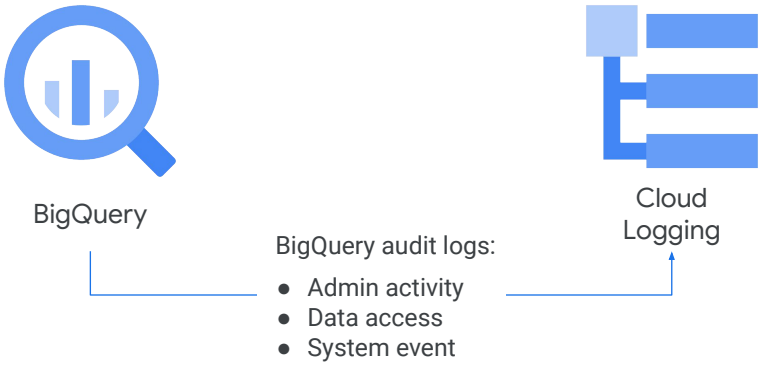
Google Cloud

Google Cloud's operations suite supports a variety of functions on all services in Google Cloud. You can monitor BigQuery using Cloud Monitoring, where various charts and alerts are defined based on BigQuery metrics. For example, you can monitor system throughput using the Query Time metric or visualize query demand trends based on the Slots Allocated metric. When you need to plan ahead for a demanding query, you can use the Slots Available metric. To stay proactive about system health, you can create alerts based on thresholds that you define.

Google Cloud's operations suite provides a self-service web-based portal. You can control access to the portal the same way that you control access to all resources within Google Cloud. BigQuery automatically creates audit logs of user actions. You can examine the logs in Cloud Logging, or export them to a BigQuery dataset in a batch or as a data stream, and use your preferred analysis tool to visualize the logs.

Proprietary + Confidential

Cloud Audit Logs can provide BigQuery-specific log information



BigQuery audit logs:

- Admin activity
- Data access
- System event

Google Cloud

Cloud Audit Logs are a collection of logs provided by Google Cloud that provide insight into operational concerns related to your use of Google Cloud services. BigQuery audit logs provide specific information about who is doing what in your BigQuery environment.

Proprietary + Confidential

BigQuery supports two versions of log messages

`AuditData` (older version)

The older AuditData is reported as the `protoPayload.serviceData` submessage within a LogEntry message.

Use this for new work

`BigQueryAuditMetadata` (newer version)

The newer BigQueryAuditMetadata is reported as the `protoPayload.metadata` within a LogEntry message. In these logs, the `protoPayload.serviceData` information is not set or used.

Google Cloud

Over time, administrators have requested additional information about what's going on inside BigQuery. As a result, there are two versions of log messages.

The newer format, BigQueryAuditMetadata, represents a better view into BigQuery operations. BigQueryAuditMetadata provides details on how resources are changed indirectly by other resources, particularly when reporting on asynchronous events and events that aren't strongly coupled to a particular API call. For example, the BigQueryAuditMetadata entries can report when BigQuery tables are removed because of a configured expiration time. This information is not available in the older logging format.

The newer BigQueryAuditMetadata is reported as the protoPayload.metadata within a LogEntry message. In these logs, the protoPayload.serviceData information isn't set or used.

The older AuditData log entries tend to map directly to individual API calls made against the BigQuery service. The older AuditData is reported as the protoPayload.serviceData submessage within a LogEntry message.

In general, most users should leverage the newer BigQueryAuditMetadata logs, but the system will continue to report the older AuditData logs to support existing users of the audit log functionality.

For an explanation of the data available within the audit logs see

https://cloud.google.com/bigquery/docs/reference/auditlogs.

Proprietary + Confidential

Audit logs are organized into three log streams

Data Access

Jobs:
`JobInsertion`
`JobChange`

Table data modifications:
`TableDataChange`
`TableDataRead`

System Event

You can set an expiration time on tables to remove them at a specified time.

A `TableDeletion` event is written to the system event stream when the table expires and is removed.

Admin Activity

All remaining activities and events are reported as part of the main activity stream.

For example:
- Job insertions
- Job completions
- Dataset creation

Google Cloud

Audit logs are provided for each product in Google Cloud. The logs are organized into three streams: Data Access, System Event, and Admin Activity.

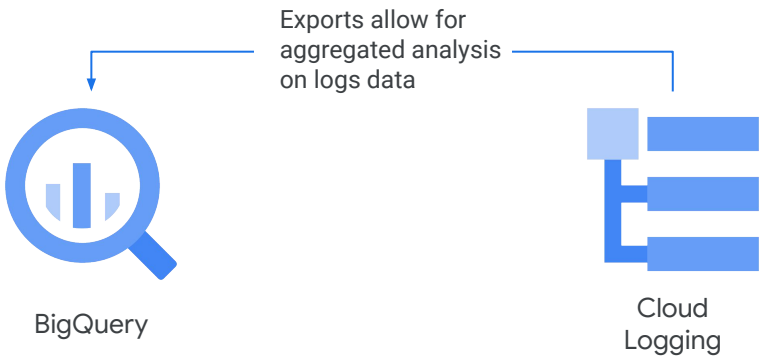
For BigQuery, the **data access** stream contains entries about jobs which are actions that BigQuery runs on your behalf. These include loading, exporting, querying, or copying data. This stream also contains entries about modifications of table data. For example, when a load job appends data to a table, a TableDataChange event is reported. When a consumer reads a table or the results of a query, a TableDataRead event is reported. Data access audit logs contain API calls that read configuration or metadata of resources, as well as user-driven API calls that create, modify, or read user-provided resource data. They don't record the data access operations on resources that are publicly shared.

System event audit logs contain log entries for Google Cloud administrative actions that modify the configuration of resources. These are generated by Google systems, they aren't driven by direct user action. For example, in BigQuery you can set an expiration time on tables to remove them at a specified time. When the table expires and is removed, a TableDeletion event is written to the system event stream because the system performed the action.

Admin activity audit logs contain log entries for API calls or other admin actions that modify the configuration or metadata of resources. This is the catch-all log stream. Events such as job insertions and completions are reported in this stream, as are other resource events such as dataset creation.

Proprietary + Confidential

Log exports from Cloud Logging to BigQuery allows for aggregated analysis on log data



Google Cloud

BigQuery automatically sends audit logs to Cloud Logging, however, these logs are only retained for 30 days and some users find querying difficult. Google Cloud logging allows users to filter and export messages to other services, including Pub/Sub, Cloud Storage, and BigQuery. For longer term retention or to facilitate querying, logs are often exported to BigQuery. Having logs in BigQuery creates a data warehouse of logs which provides the ability to do aggregated analysis on logs data.

Proprietary + Confidential

To filter relevant BigQuery Audit messages, you can express filters as part of the export

```
protoPayload.metadata."@type"="type.googleapis.com/google.cloud.audit.
BigQueryAuditMetadata"
```

Google Cloud

To filter relevant BigQuery Audit messages, you can express filters as part of the export. For example, this advanced filter represents an export that only includes the newer BigQueryAuditMetadata format.

You can express additional filters based on the fields within the log messages.

Proprietary + Confidential

Querying exported logs: Report expired tables

```
#standardSQL
  SELECT
    protopayload_auditlog.resourceName AS resourceName,
    receiveTimestamp as logTime
  FROM
`MYPROJECTID.MYDATASETID.cloudaudit_googleapis_com_system_event_201901*`
  WHERE
    protopayload_auditlog.methodName = 'InternalTableExpired'
  ORDER BY resourceName
```

Google Cloud

Once the data is exported to BigQuery, users can use standard SQL to query the logs.

This example, and the one that I'll show you next, use the newer BigQueryAuditMetadata messages. Because schema conversion is done during the export from Cloud Logging into BigQuery, the message bodies are presented in semi-structured form.

This sample query returns the table name and timestamp of tables deleted because their expiration time was reached.

Note that you must change the FROM clause in each of these examples to the corresponding exported tables in your project.

Proprietary + Confidential

Querying exported logs: Most popular datasets

```
#standardSQL
  SELECT
    REGEXP_EXTRACT(protopayload_auditlog.resourceName, '^projects/[^/]+/datasets/([^/]+)/tables') AS datasetRef,
    COUNT(DISTINCT REGEXP_EXTRACT(protopayload_auditlog.resourceName,
'^projects/[^/]+/datasets/[^/]+/tables/(.*)$')) AS active_tables,
    COUNTIF(JSON_EXTRACT(protopayload_auditlog.metadataJson, "$.tableDataRead") IS NOT NULL) AS dataReadEvents,
    COUNTIF(JSON_EXTRACT(protopayload_auditlog.metadataJson, "$.tableDataChange") IS NOT NULL) AS
dataChangeEvents
  FROM `MYPROJECTID.MYDATASETID.cloudaudit_googleapis_com_data_access_2019*`
  WHERE
    JSON_EXTRACT(protopayload_auditlog.metadataJson, "$.tableDataRead") IS NOT NULL
    OR JSON_EXTRACT(protopayload_auditlog.metadataJson, "$.tableDataChange") IS NOT NULL
  GROUP BY datasetRef
  ORDER BY dataReadEvents DESC
```

Google Cloud

This query shows coarse, per-dataset statistics about table reads and table modifications.

The protopayload_auditlog.metadataJson is a STRING field that contains the JSON representation of the message. You can leverage JSON functions in standard SQL to analyze this content. In this query, the JSON_EXTRACT function is used to quickly parse the hierarchical JSON string.

Proprietary + Confidential

Timeout errors have two main causes

- Query is malformed or inefficient
- Query being executed has WITH statements

> 6 hours

Google Cloud

In cases where query execution exceeds 6 hours, the query will fail due to a timeout error. The causes for queries that generate this failure generally fall into one of two categories.

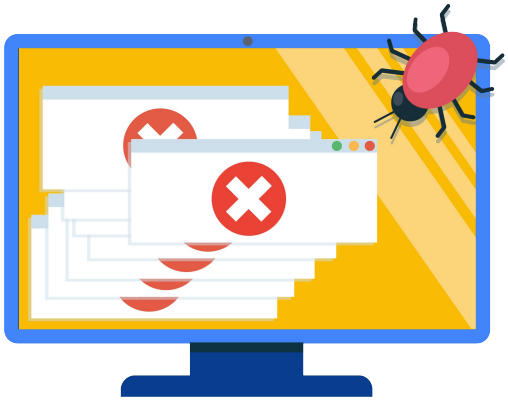
If a query is malformed, it will get an immediate error because the query cannot even run. On the other hand, if the query is inefficient, it will run, but may never produce a result.

One reason a query may be inefficient is because it is has many WITH statements that are increasing the overall runtime of the query.

If the query has many WITH statements, it may be necessary to break it up into multiple query executions. Note that the 6-hour timeout applies to the entire query executed. If a large query is broken up into multiple smaller queries, each individual query instead works within the 6-hour timeout period.

Proprietary + Confidential

Internal errors don’t always provide the underlying context so they can be confusing to debug



Google Cloud

Internal errors don’t always provide the underlying context, so they can be confusing to debug.

An internal error actually means that BigQuery has encountered an error it hasn’t seen before. Generally these indicate a bug in the system. The best remediation to this error is to open a support ticket or file a bug.

Proprietary + Confidential

Resources Exceeded errors don't relate to the number of slots available

```
#standardSQL
SELECT repo_name, ref, path, count(id) as item_count
FROM `bigquery-public-data.github_repos.files`
GROUP BY repo_name, ref, path
ORDER BY item_count desc
LIMIT 100000000
```

Google Cloud

Contrary to what most assume at first, resources-exceeded errors aren't related to the number of slots available. Slots determine throughput, but not "resources" as in the 'Resources Exceeded' error. If you use all the slots available, the queries will just run slower. Resources Exceeded occurs in the case where one slot (worker) somewhere in the system ran out of memory during query execution. This usually happens because the data that the query references was partitioned (dynamically) such that too many rows landed on one worker.

A common reason for this is a "large sort" whether in an ORDER BY clause or in a WINDOW function, for example PARTITION by x ORDER BY y. These errors can be resolved by adding a properly defined LIMIT to the ORDER BY clause or by changing your partition logic so that records are more evenly distributed within the windows. In the example, a limit of 100 million is in place but given that the underlying table has over 2 billion rows such a limit is inadequate. A very low limit such as 10,000 would likely be more appropriate since the goal of this query is to sort by "item_count" in descending order.

Another cause of Resources Exceeded errors is that the data is skewed, for example, joins or aggregates on a field where one value accounts for a disproportionate number of rows. This often happens where default values, such as NULL, 0, or 12-31-9999 account for a large number of rows.

Proprietary + Confidential

Results too large error occurs if the response back is larger than 128MB compressed

- Use filters to limit the result set
- Use the LIMIT clause
- Specify a destination table with allowLargeResults

Google Cloud

BigQuery limits result sets to approximately 128MB compressed. Queries returning large results cannot fit into the response sent back from the BigQuery API server. This error is commonly thrown on queries that return a large volume of data, such as a SELECT over a huge table or ETL-style queries that normalize data without reduction/aggregation.

These errors can be solved in one of three ways:

- Use filters to limit the result set,
- Use the LIMIT clause, or
- Specify a destination table with allowLargeResults.

Proprietary + Confidential

Agenda

BigQuery Concepts

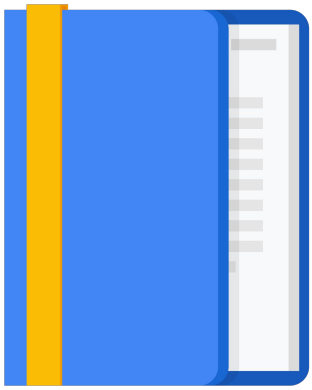
BigQuery Permissions and Security

Monitoring and Auditing

Schema Design

Partitioning and Clustering

Data Capture and Load Jobs

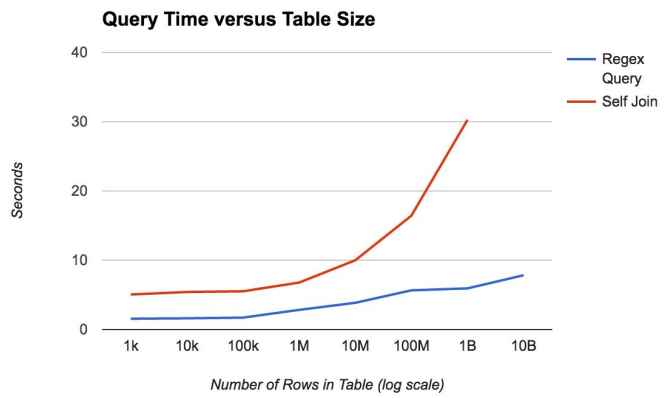


Google Cloud

As we discussed earlier, BigQuery stores repeated, nested fields in a columnar format. Let's examine how to optimize BigQuery performance by taking advantage of enhanced schema design options.

Proprietary + Confidential

Query performance on very large tables can be improved through denormalization



Google Cloud

The BigQuery analytical engine supports data manipulation language (DML) actions, but it isn't meant to be used for online transaction processing (OLTP) which favors writing but for online analytical processing (OLAP) which is optimized for reading.

When reading data, JOIN is among the highest resource-consuming operators. Earlier, I mentioned that BigQuery stores repeated, nested fields in a columnar format. This structure facilitates denormalizing data to reduce or eliminate JOINs.

Proprietary + Confidential

Disadvantages of denormalization

- Denormalized schemas aren't storage-optimal.
  (Low cost of BigQuery storage addresses concerns about storage inefficiency though.)

- Maintaining data integrity can require increased machine time and sometimes human time for testing and verification.

Google Cloud

The concept of denormalization was devised in a time when storage was very expensive, so it was worthwhile to find ways to minimize the amount of data stored at the expense of computing resources.

In traditional RDBMS databases, denormalized schemas aren’t storage-optimal because columns of data from the parent table must be repeated in each row of the child table.

And to ensure integrity, traditional relational databases use foreign keys. These require effort to set up and create overhead when records are inserted, updated or deleted.

Proprietary + Confidential

For denormalizing data, use nested/repeated fields rather than writing facts into a flat table structure

```
{
  "orderID": "ORDER",
  "custID": "EMAIL",
  "custName": "NAME",
  "timestamp": "TIME",
  "location": "LOCATION",
  "purchasedItems": [
    {
      "sku": "SKU",
      "description": "DESCRIPTION",
      "quantity": "QTY",
      "price": "PRICE"
    },
    {
      "sku": "SKU",
      "description": "DESCRIPTION",
      "quantity": "QTY",
      "price": "PRICE"
    }
  ]
}
```

Google Cloud

The conventional method of denormalizing data involves writing a fact, along with all its dimensions, into a flat table structure. For example, for sales transactions, you would write each fact to a record, along with the accompanying dimensions, such as order and customer information.

In contrast, denormalizing data takes advantage of BigQuery's native support for nested and repeated structures. Expressing records using nested and repeated structures provides a more natural representation of the underlying data.

In this example, the outer part (or parent) part of the structure contains the order and customer information, and the inner part of the structure contains the individual line items of the order, which are represented as nested, repeated elements.

Using schemas with nested and repeated fields simplifies loading JSON, Avro and Parquet files, but can also be used to optimize performance of data imported from an OLTP system that uses traditional normalized relational database structure.

Proprietary + Confidential

General guidelines to design the optimal schema for BigQuery

- Denormalize a dimension table that is larger than 10 gigabytes, unless you see strong evidence that data manipulation, `UPDATE` and `DELETE` operation, costs outweigh the benefits of optimal queries.
- Keep a dimension table that is smaller than 10 gigabytes normalized, unless the table rarely goes through `UPDATE` and `DELETE` operations.
- Take full advantage of nested and repeated fields in denormalized tables.

Google Cloud

The potential increase in performance for denormalizing might not outweigh the effort required to modify ingress processes and existing queries. So here are a few general guidelines to consider when designing the optimal schema for BigQuery.

- In some cases, denormalizing won't improve performance. For example, if your dimensional table is relatively small, it may be best to maintain it as a separate table. You should also consider the impact of UPDATE and DELETE operations on your design. We'll discuss slowly changing dimensions in an upcoming lesson.
- While not a hard-limit, 10 GB is a good boundary for consideration of the question "to normalize or not to normalize". But don't forget to consider UPDATEs and DELETES.
- A best practice is to "use nested and repeated fields where appropriate" but what does that mean? To make an informed decision, consider the size of your fact and dimension tables along with the frequency and volume of UPDATES and DELETES. As car manufacturers say, "Your mileage may vary", so be sure to test performance before making massive structural changes.

Proprietary + Confidential

Agenda

BigQuery Concepts

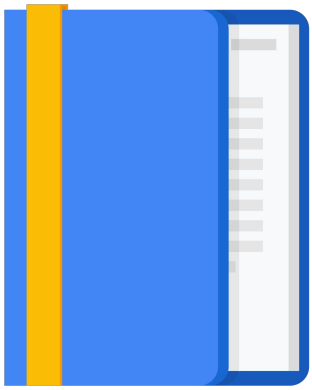
BigQuery Permissions and Security

Monitoring and Auditing

Schema Design

Partitioning and Clustering

Data Capture and Load Jobs



Google Cloud

Another method for improving performance in BigQuery is to make access to data more efficient using partitions and clustering.

Proprietary + Confidential

BigQuery supports three ways of partitioning tables

Ingestion time

```
bq query --destination_table mydataset.mytable
--time_partitioning_type=DAY
...
```

Any column that is of type DATE or TIMESTAMP

```
bq mk --table --schema a:STRING,tm:TIMESTAMP
--time_partitioning_field tm
```

Integer-typed column

```
bq mk --table --schema "customer_id:integer,value:integer"
--range_partitioning=customer_id,0,100,10 my_dataset.my_table
```

Google Cloud

Table partitioning is a method of dividing the table into sections based on user-defined criteria. Queries that filter on the partitioning column benefit tremendously from this technique. BigQuery offers three partitioning options.

The first option for partitioning is based on the time the data was inserted (or ingested) into BigQuery. This works well for data that is added in real time but doesn't work well when loading historical data.

As a result, the most common partitioning choice is using a column of data such as OrderDate or EventDate. We'll talk about optimizing the loading of this type of data later.

For systems that aren't date-based, BigQuery also provides the option to partition on an integer column, such as CustomerID.

Proprietary + Confidential

Partitioning can improve query cost and performance by cutting down on data being queried

```
SELECT
  field1
FROM
  mydataset.table1
WHERE
  _PARTITIONTIME > TIMESTAMP_SUB(TIMESTAMP('2016-04-15'), INTERVAL 5 DAY)
```

Isolate the partition field in the left-hand side of the query expression!

```
bq query \
--destination_table mydataset.mytable
--time_partitioning_type=DAY --require_partition_filter
...
```

Google Cloud

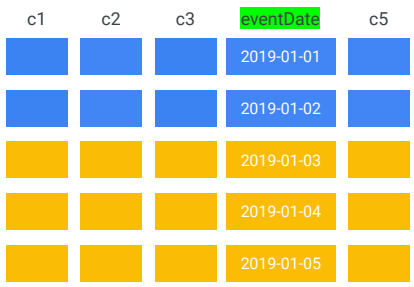
To benefit from partitioning, queries must include the partitioning column in the WHERE clause as shown in the first query.

To ensure that users include the partitioning column in their query, you can define the table to require a filter as shown in the table definition.

To ensure that the query processor recognizes that it should look only at designated partitions, make sure the partition column stands alone on the left in the row filter. Note that there can be more conditions in the WHERE clause.

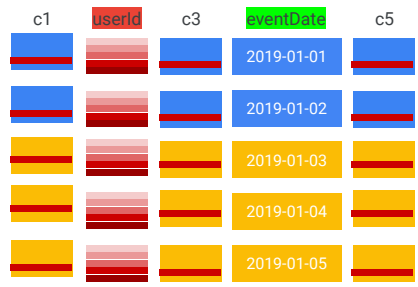
Proprietary + Confidential

BigQuery automatically sorts the data based on values in the clustering column(s)



SELECT c1, c3 FROM ...
WHERE eventDate BETWEEN “2019-01-03” AND “2019-01-05”

Partitioned tables



SELECT c1, c3 FROM ... WHERE userId BETWEEN 52 and 63 AND eventDate BETWEEN “2019-01-03” AND “2019-01-05”

Clustered tables

Google Cloud

Once a table is partitioned, it can also be clustered. Clustering provides a mechanism for sorting the data within a partition based on the clustering columns. If you specify multiple columns for clustering, they’ll be sorted starting with the first column you specify. As with partitioning, clustering will facilitate queries that include the clustering column or columns in the WHERE clause but may also improve performance of queries with an ORDER BY.

The example on the left shows the benefit of partitioning. The query processor will ignore the partitions for January 1 and January 2 completely so less data is processed.

Clustering has been added to the table on the right. Again, only the partitions in yellow are considered, but within each partition the query data is sorted by userID so the query processor finds 52 then proceeds to 63 and quits.

Proprietary + Confidential

When to use clustering

- Your data is already partitioned on a `DATE` or `TIMESTAMP` column.
- You commonly use filters or aggregation against particular columns in your queries.

Google Cloud

Currently, you can define a table with partitions and not include clustering, but you cannot have clustering without partitioning.

- Table clustering is supported for both ingestion time partitioned tables and for tables partitioned on a DATE or TIMESTAMP column. Currently, clustering is not supported for non-partitioned tables.
- When you use clustering and partitioning together, the data can be partitioned by a date or timestamp column and then clustered on a different set of columns. In this case, data in each partition is clustered based on the values of the clustering columns. The query processor considers partitions when providing cost estimates for queries.

Proprietary + Confidential

When you use the clustered table feature, you are subject to the limits on partitioned tables

Partitioned tables limits	
Max partitions per partitioned table:	4,000
Max partitions modified by a single job:	2,000
Max partition modifications per day per table:	5,000
Maximum rate of partition operations:	50 every 10 sec

Google Cloud

There are some limitations on partitioning and clustering as shown here. This isn't an exhaustive list and these numbers may change as BigQuery evolves, so always refer to the online documentation for the latest information.

Proprietary + Confidential

Agenda

BigQuery Concepts

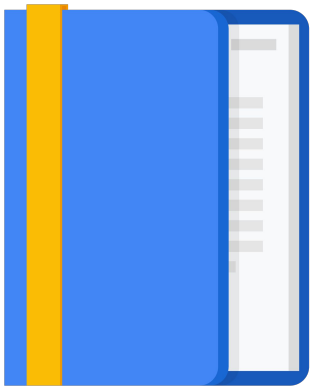
BigQuery Permissions and Security

Monitoring and Auditing

Schema Design

Partitioning and Clustering

Data Capture and Load Jobs

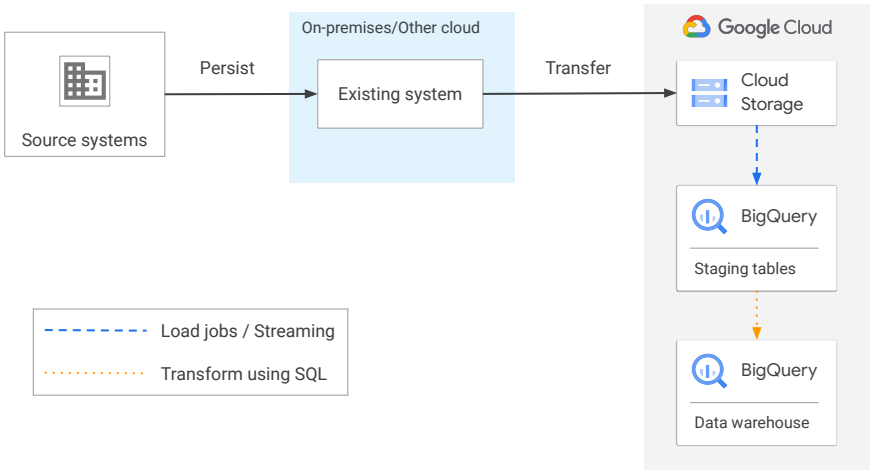


Google Cloud

A data warehouse is only as good as the data it contains, so it's important to ensure the data is fresh by adding and updating the data warehouse from operational sources. Let's take a high level look at how you might be able to implement that functionality in Google Cloud. In later modules, we'll examine these in more detail.

Proprietary + Confidential

Loading data: Transform using ELT

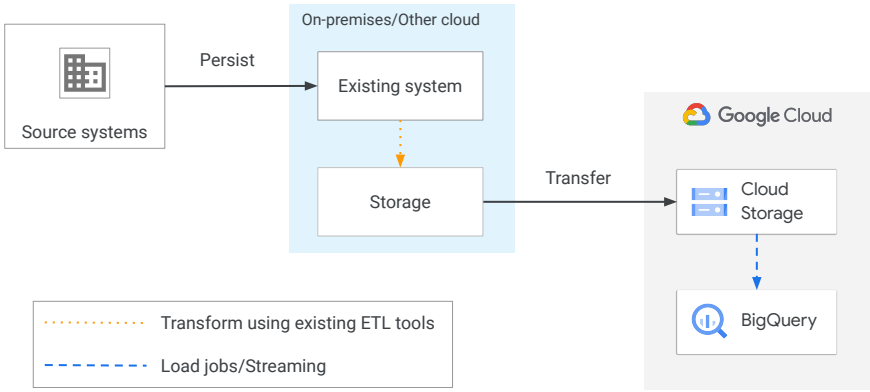


Google Cloud

One method of processing source data is Extract-Load-Transform (ELT). This example shows **E**xtracting data from the source system in the schema that is native to the existing on-premises data storage and transfering the files to Cloud Storage. The data is then **L**oaded into a set of staging tables in BigQuery and then **T**ransformed into the ideal schema for BigQuery by using BigQuery SQL commands.

Proprietary + Confidential

Loading data: Transform using existing ETL tools

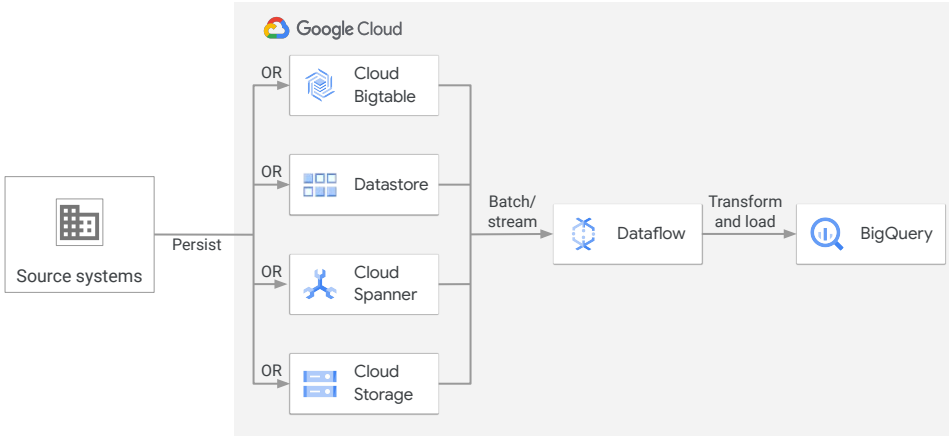


Google Cloud

Another ingestion method is Extract-Transform-Load (ETL). This methodology **E**xtracts and **T**ransforms the data prior to transferring to Cloud Storage so that it can be **L**oaded into BigQuery using load jobs or by streaming.

Proprietary + Confidential

Expanding your footprint in Google Cloud provides more options



Google Cloud

Google Cloud supports a variety of data storage services, so as you migrate more functionality to the cloud you can choose the appropriate storage for each process. There's also a variety of tools to facilitate the process of ingesting new and updated data. These are discussed in the **ETL Tools and Positioning** module.

Proprietary + Confidential

Batch load supports different file formats

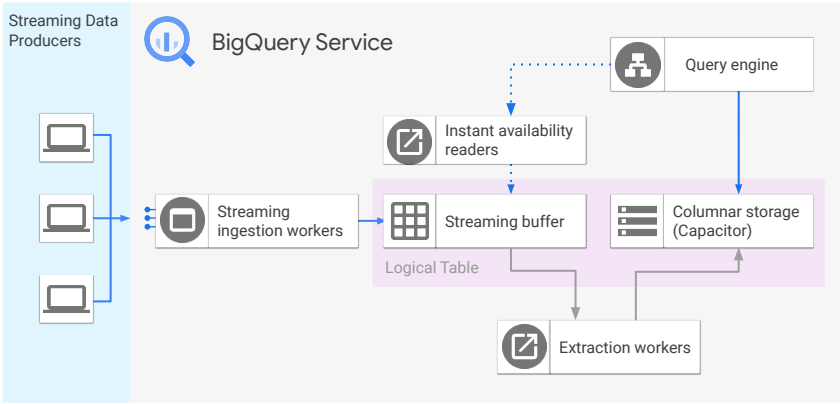
- CSV
- NEWLINE_DELIMITED_JSON
- AVRO
- DATASTORE_BACKUP
- PARQUET
- ORC

Google Cloud

Raw data can be produced in many formats. Traditional CSV files are being replaced by more efficient formats such as AVRO and Parquet. In most cases, BigQuery can automatically detect the schema of the file and create tables for you. BigQuery sets daily limits on the number and size of load jobs that you can perform per project and per table. In addition, BigQuery sets limits on the sizes of individual load files and records. Don’t worry if your needs exceed these quotas; in the ETL Tools and Positioning module, we’ll look at other methods to ingest data.

Proprietary + Confidential

For an alternate and complementary approach, you can also stream data directly into BigQuery

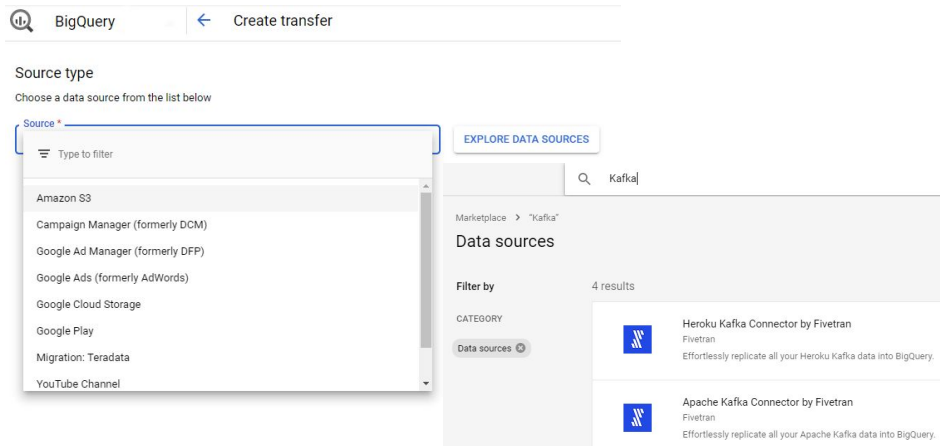


Google Cloud

Historically, data has been loaded in batches. However, use cases such as credit card fraud detection demand real-time data which requires streaming data into the data warehouse. This data is available immediately for dashboards and analytics. Google Cloud supports several methods for processing streaming data and these are discussed in greater detail in the **Streaming Analytics** module.

Proprietary + Confidential

BigQuery Data Transfer Service



Google Cloud

If your goal is simply to migrate data, you can use the BigQuery Data Transfer Service. It automates data movement from SaaS applications and on-premise or cloud-based data warehouses to BigQuery on a scheduled, managed basis. It can also be used to efficiently move data between regions. In the **Migrating to BigQuery** module, we'll use this service to migrate from Teradata to BigQuery.

Proprietary + Confidential

Agenda

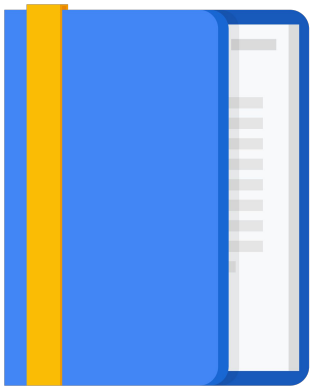
Handling Change and Slowly Changing Dimensions

Querying Data

Lab: Exploring Your Ecommerce Dataset with SQL in BigQuery

Managing Workloads and Concurrency

Analyzing Data



Google Cloud

As you know, data warehouses are comprised of facts and dimensions. While facts generally remain the same, the attributes of dimensions may change, so you need to address the nature and scope of the changes.

Proprietary + Confidential

BigQuery manages service availability, but you control the duration and timeliness of your datasets

All table modifications are ACID compliant

But timeliness might be affected by:

- Periodic loads versus streaming ingest
- Priority of load jobs versus analytics jobs

Google Cloud

Many data warehouses operate under strict Service Level Agreements (SLAs), demanding little to no downtime. While Google handles BigQuery's uptime, customers control the availability and responsiveness of their datasets with their approach to reflecting change in the data.

All table modifications in BigQuery are ACID compliant so modifying a table doesn't require downtime. However, a customer’s internal process might require a testing and validation phase before making newly refreshed data available for analysis. Also, because DML operations compete against analytical workload over slots, they might prefer to isolate them. For these reasons, you might introduce downtime.

Fortunately you can apply most of the old and proven techniques used in traditional data warehouse environments for handling analysis downtime when changes are required.

Proprietary + Confidential

A sliding time window means there's no need to delete older data

In BigQuery, you can afford to retain older data:

- Use sliding windows based on partition field
- Take advantage of BigQuery long-term storage pricing

If necessary to delete older data, set up automatic table/partition expiry

Google Cloud

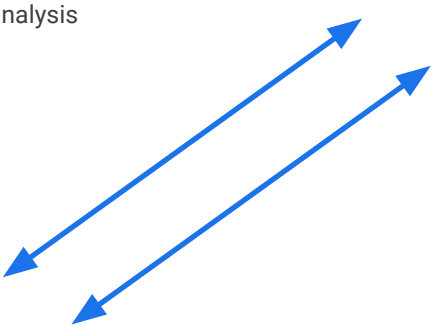
Traditional data warehouses generally limit historical data due to storage and performance constraints. Because BigQuery uses the latest technology and is designed to scale, the size of a data warehouse is virtually unlimited. By keeping the entire history, customers can deliver more insight on their business.

If customers are concerned about query efficiency, you can use partitioning as I discussed earlier to limit the amount of data processed. If cost is an issue don't worry - BigQuery automatically gives a 50% discount on data stored more than 90 days without modification. If customers prefer not to keep older data but don't want the hassle of managing it you can set up automatic expiration at the table or partition level and BigQuery will get rid of the old data for them.

Proprietary + Confidential

For the most part, schema changes are scheduled as version upgrades

Design, develop, and test the upgrade in parallel while the previous version of the data warehouse is serving analysis workloads



Google Cloud

Data warehouses are constantly evolving as users want new and different functionality and data. As a result you need to be able to change both schemas and data without impacting users (remember our SLA!)

Generally, a change in the schema is accompanied by changes to ingestion processes, reports, and other components of the data warehouse system. As a result, they're considered a new version. In a traditional environment, you design, develop, and test the upgrade in parallel while the previous version of the data warehouse is serving the analysis workloads. You follow the same approach in applying schema changes to a BigQuery data warehouse.

Proprietary + Confidential

SCD Type 1: Overwrite attribute value

Before:

PRD_SK	PRD_ID	PRD_DESC	PRD_CATEGORY
123	ABC	awesome moisturizer cream - 100 oz	health and beauty

After:

PRD_SK	PRD_ID	PRD_DESC	PRD_CATEGORY
123	ABC	awesome moisturizer cream - 100 oz	~~health and beauty~~ cosmetics

Google Cloud

But what about changes to data? Typically facts don't change, but dimensional attributes might. The ANSI standard defines several types of slowly changing dimensions, but the most commonly used are type 1 and type 2. In a type 1 slowly changing dimension, the value of the attribute, in this case PRD_CATEGORY, is simply replaced with the new value. This is the simplest method, but provides no means to track the historical value.

A Type 1 change is simple if the attribute is in a normalized dimension table, particularly if the table and the number of records updated are small. An UPDATE statement will only impact analytical users during the time it's executed, which should be very short.

If the dimensional data is nested the impact will be greater, so you should consider some of the other options we'll discuss shortly.

Proprietary + Confidential

SCD Type 2: Change attribute value and maintain history

Before:

PRD_SK	PRD_ID	PRD_DESC	PRD_CATEGORY	START_DATE	END_DATE
123	ABC	ace moisturizer cream - 100 oz	health and beauty	31-Jan-2009	NULL

After:

PRD_SK	PRD_ID	PRD_DESC	PRD_CATEGORY	START_DATE	END_DATE
123	ABC	ace moisturizer cream - 100 oz	health and beauty	31-Jan-2009	18-JUL-2017
124	ABC	ace moisturizer cream - 100 oz	cosmetics	19-JUL-2017	NULL

Fact table:

TRANSACTION_DATE	PRD_SK	PRD_ID	PRD_DESC	PRD_CATEGORY	UNITS	AMOUNT
18-JUL-2017	123	ABC	ace moisturizer cream - 100 oz	health and beauty	2	25.16
19-JUL-2017	124	ABC	ace moisturizer cream - 100 oz	cosmetics	1	13.50

Google Cloud

While a Type 1 slowly changing dimension is fine for tracking some changes, you must use Type 2 if you need to know historical values. As you can see, implementing Type 2 requires additional columns in the dimension table to identify “when” the value applies. This history allows categorization of sales of product ABC to the “health and beauty” category before July 19, 2017, and to the “cosmetics” category after that date.

Proprietary + Confidential

When it comes to SCD, there's no one-size-fits-all solution

- In a denormalized table:
  - View switching
  - In-place partition loading
- Isolate changes by using normalized data schema and isolate change in small dimension tables
  - Update data masking

Google Cloud

I spoke about normalization versus denormalization earlier and mentioned that you should consider UPDATES when deciding. A normalized schema minimizes the impact of Slowly Changing Dimensions by isolating the change in the dimension tables, however, it may impact query performance so you must consider both when designing the schema.

There are several options for incorporating changes to dimensions and they depend on the schema and the number of changes.

Proprietary + Confidential

Technique 1: View switching

Two views or copies of data

- Analysts query main view
- Shadow view is updated
- Main view swapped with shadow

Considerations:

- Zero down-time
- DML on main view might be problematic
- Apply view-switching to custom partitions

Google Cloud

View switching is a technique whereby the analysts use a “main” view that’s pointing to the current table. The update process creates and loads the “shadow” table with the original values plus any updates. The shadow table can be validated without impacting users who are using the view that points to the current table. Once the “shadow” table is verified, the view is modified to point to the “shadow” table instead of the original current table.

The view could be based on a single table that included a column such as view_type to differentiate the “current” and “shadow” rows. Once the “shadow” rows have been validated, the view could be changed to use the “shadow” view_type in the filter. This method isn’t recommended because DML operations against the "shadow" view of the table could slow down user queries against the "main" view without offering any real benefits.

While view switching offers zero analysis downtime, it has a higher cost because during the update cycle, two copies of the data exist. More importantly, if update cycles happen at a higher rate than 90 days, this approach could prevent an organization from taking advantage of long-term storage pricing.

Sometimes, different segments of data change at their own pace. For instance, sales data in North America is updated on a daily basis, while data for Asia Pacific is updated on a biweekly basis. In such situations, it’s best to partition the table based on the driving factor for the change, Country in this example. View switching is then applied to the impacted partitions and not the entire data warehouse. Currently, you

can only partition based on a custom data attribute, such as Country, if you use an integer column such as CountryID, or by explicitly splitting the data into multiple tables.

Proprietary + Confidential

Technique 2: In-place partition loading

To replace data in a target partition with data from a query of another table:

```
bq query --use_legacy_sql=false --replace \
         --destination_table 'flight_data.fact_flights_part$20140910' \
         'select * REPLACE(...) from `ods.load_flights_20140910`
```

To replace data in a target partition by loading from Cloud Storage:

```
bq load  --replace \
         --source_format=NEWLINE_DELIMITED_JSON
         'flight_data.fact_flights_part$20140910' \
         gs://{bucket}/load_flights_20140910.json
```

Google Cloud

When the change in data can be isolated by a partition and brief analysis downtime is tolerated, view switching might be overkill. Instead, data for the affected partition can be staged in another BigQuery table or exported to files in Cloud Storage, where it can be replaced during analysis downtime or even in real time if the partition is relatively small. Because the replace is an atomic operation, users won't see any new data until all data in the partition has been replaced.

Proprietary + Confidential

Technique 3: Update data masking

```
SELECT f.order_id as order_id, f.customer_id as customer_id,
    IFNULL(u.customer_first_name, f.customer_first_name) as customer_first_name,
    IFNULL(u.customer_last_name, f.customer_last_name) as customer_last_name
FROM fact_table f
LEFT OUTER JOIN pending_customer_updates u
ON f.customer_id = u.customer_id
```

Google Cloud

A small and frequently changing dimension is a prime candidate for normalization. In this technique, updates to the dimension are stored in an isolated table or view that is conditionally joined with the rest of the data. This technique will impact query performance. The amount of performance degradation depends on the size of the pending_customer_updates table.

Proprietary + Confidential

Agenda

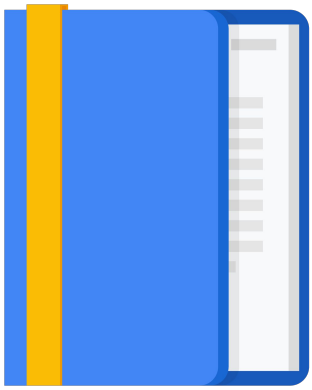
Handling Change and Slowly Changing Dimensions

Querying Data

Lab: Exploring Your Ecommerce Dataset with SQL in BigQuery

Managing Workloads and Concurrency

Analyzing Data



Google Cloud

The main purpose of a data warehouse is to serve as a repository that end users can query to get the information they need, so it's important to know how to query data efficiently. There's a lesson on Query Optimization coming up, so we'll focus on writing and evaluating queries in this lesson.

Proprietary + Confidential

BigQuery supports standard SQL queries

```
SELECT
  name, count(1) as num_repos
FROM
  `bigquery-public-data`.github_repos.languages, UNNEST(language)
GROUP BY name
ORDER BY num_repos
DESC limit 10
```

Google Cloud

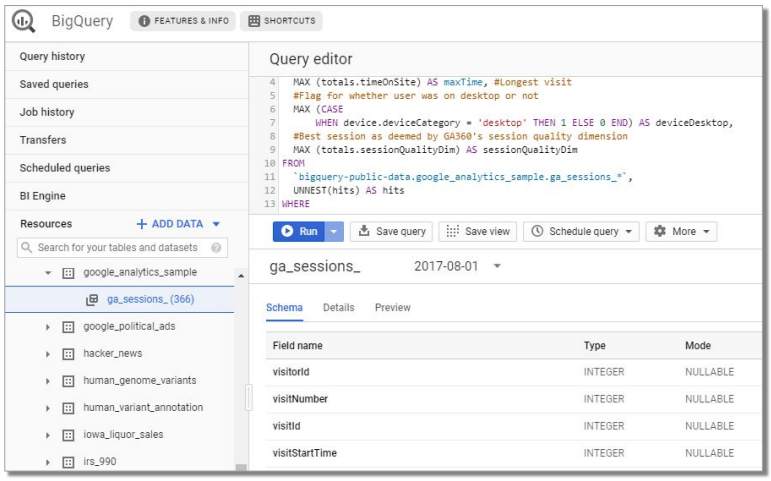
BigQuery supports standard SQL queries and is compatible with ANSI SQL 2011. BigQuery SQL reference provides a comprehensive description of all functions, operators, and regex capabilities that are supported.

Note that prior to supporting standard SQL, BigQuery supported an alternate SQL version that's now referred to as Legacy SQL. I recommend using the updated SQL standard in your queries.

Because BigQuery supports nested and repeated fields as part of the data model, its SQL support has been extended to specifically support these field types. For example, using the GitHub repository languages public dataset, you could issue the UNNEST command, which lets you iterate over a repeated field, in this case "language".

Proprietary + Confidential

BigQuery web UI allows interactive dataset querying

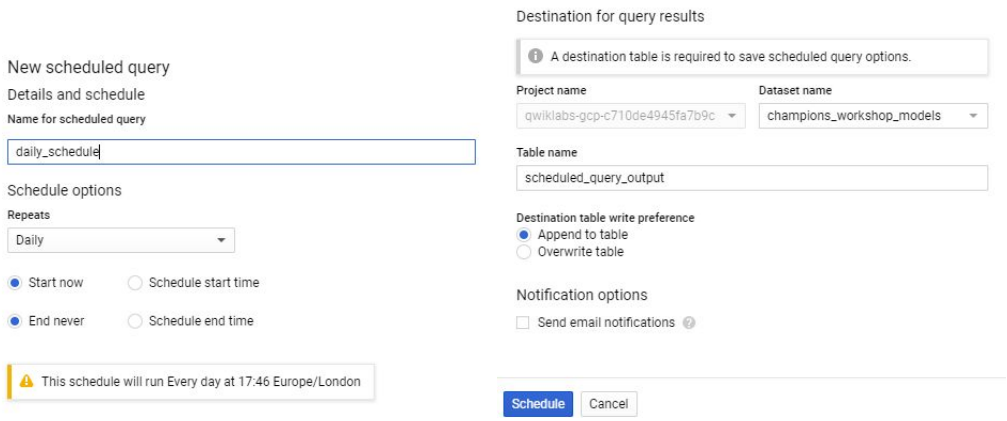


Google Cloud

The BigQuery web UI allows interactive querying of datasets and provides a consolidated view of datasets across projects that you have access to. The console also provides several useful features, such as saving and sharing ad-hoc queries, tuning and editing historical queries, exploring tables and schemas, and gathering table metadata.

Proprietary + Confidential

Automate the execution of queries based on a schedule



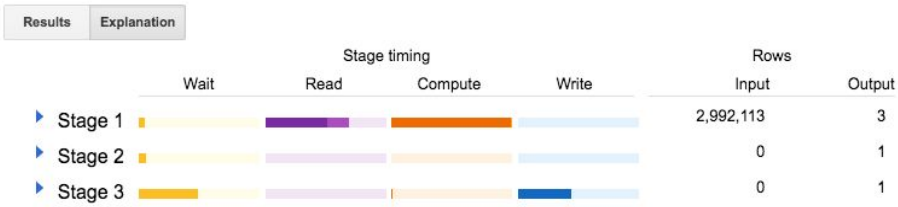
Google Cloud

To maximize performance and minimize cost you may want to automate the execution of queries based on a schedule/event, and cache the results for later consumption.

You can schedule queries to run on a recurring basis. Scheduled queries must be written in standard SQL, which can include Data Definition Language (DDL) and Data Manipulation Language (DML) statements. The query string and destination table can be parameterized, allowing you to organize query results by date and time.

Proprietary + Confidential

Query optimization

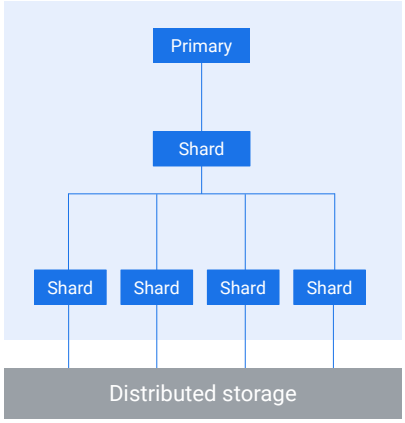


Google Cloud

Each time BigQuery executes a query, it executes a full-column scan. BigQuery doesn't use or support indexes. Because BigQuery performance and query costs are based on the amount of data scanned during a query, design your queries so that they reference only the columns that are relevant to the query. When using date-partitioned tables, ensure only the relevant partitions are scanned. You can achieve this by using partition filters based on PARTITIONTIME or PARTITIONDATE.

Proprietary + Confidential

Simple query execution




```
SELECT COUNT(*) FROM
wikipedia_benchmark.Wiki1B
WHERE title LIKE "G%o%o"
```

Stage 2: Sum (1 slot)

Stage 1: Filter, Count (220 slots)

Google Cloud

Let’s go through a simple query execution.

This query has a filter and a count aggregation over a billion-record table. How does this work?

Stage 1: BigQuery applies the filter and does partial counts in parallel.

Stage 2: BigQuery computes the sum of those partial counts.

We’ll discuss what the slot numbers mean and where we get them in a second.

Proprietary + Confidential

Simple query execution: Explain plan

	Stage timing				Rows	
	Wait	Read	Compute	Write	Input	Output
Stage 1					1,249,541,131	220
Stage 2					220	1

```
Stage 1
     READ $1:title
          FROM bigquery-samples.wikipedia_benchmark.Wiki1B
          WHERE like($1, 'G%o%o')
AGGREGATE $20 := COUNT_STAR()
    WRITE $20
          TO __stage1_output

Stage 2
     READ $20
          FROM __stage1_output
AGGREGATE $10 := SUM_OF_COUNTS($20)
    WRITE $10
          TO __output
```

```
SELECT COUNT(*) FROM
wikipedia_benchmark.Wiki1B
WHERE title LIKE "G%o%o"
```

Google Cloud

So this is what the query plan looks like for that query. It may look a little different than the query text you wrote because this is what the query looks like to the execution engine.

Each part of the query is broken down into stages.

In Stage 1, you read from the underlying table and perform the filter. For this filter you can push it into the storage layer for extra fast performance. You can tell that because the filter is performed in the READ operation. Then you do the partial aggregation of the count star and write that to the next stage.

In Stage 2, you read from the previous stage output and compute the sum of the partial counts.

Finally, you return the total sum.

Proprietary + Confidential

Simple query execution: More data

	Stage timing				Rows	
	Wait	Read	Compute	Write	Input	Output
Stage 1					10,677,046,566	2,136
Stage 2					2,136	1

```
SELECT COUNT(*) FROM
wikipedia_benchmark.Wiki10B
WHERE title LIKE "G%o%o"
```

Stage 1					106,770,465,660	21,355
Stage 2					21,355	1

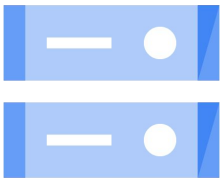
```
SELECT COUNT(*) FROM
wikipedia_benchmark.Wiki100B
WHERE title LIKE "G%o%o"
```

Google Cloud

The Wikipedia benchmark public dataset contains tables of various sizes that you can use for benchmarking. Note that the number shown in the “Output” column can an indication of the number of slots used. The default quota on slots is 2,000 so you would need to request a quota increase to get this many slots.

Proprietary + Confidential

Using federated data sources has performance implications



Cloud Storage

Google Drive

Cloud Bigtable

Google Cloud

You can run queries on data that exists outside of BigQuery by using federated data sources, but this approach has performance implications. Use federated data sources only if the data must be maintained externally. You can also use query federation to perform ETL from an external source to BigQuery. This approach allows you to define ETL using familiar SQL syntax. The federated data sources supported by BigQuery is constantly increasing so check the current documentation to see what's available. If there's a data source you really need that isn't currently provided be sure to tell your Google rep so that it can be evaluated and potentially added. It may even be on the list already and your rep can give you a timeframe for availability.

Proprietary + Confidential

BigQuery supports user-defined functions in SQL, JavaScript and scripting

Query editor

```
CREATE TEMP FUNCTION multiplyInputs(x FLOAT64, y FLOAT64)
RETURNS FLOAT64
LANGUAGE js AS """
  return x*y;
""";
WITH numbers AS
  (SELECT 1 AS x, 5 as y
  UNION ALL
  SELECT 2 AS x, 10 as y
  UNION ALL
  SELECT 3 as x, 15 as y)
SELECT x, y, multiplyInputs(x, y) as product
FROM numbers;
```

Run | Save query | Save view | Schedule query | More

Query complete (2.5 sec elapsed, 0 B processed)

Job information | Results | JSON | Execution details

Row	x	y	product
1	1	5	5.0
2	2	10	20.0
3	3	15	45.0

Google Cloud

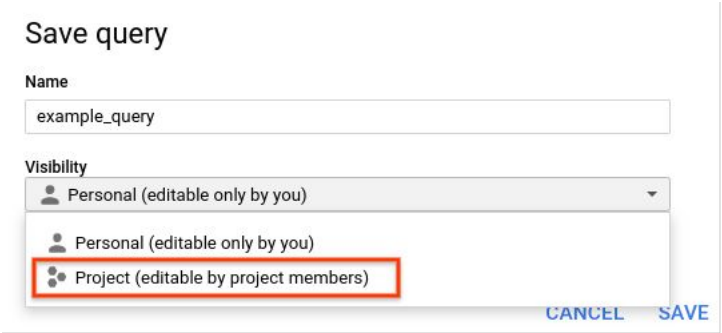
BigQuery supports user-defined functions (UDFs). A UDF enables you to create a function using another SQL expression or JavaScript.

JavaScript UDFs are temporary. This means you can only use them for the current query or command-line session. Permanent functions, scripts, and procedures are currently available only for SQL.

UDFs allow you to extend the built-in SQL functions; they take a list of values, which can be arrays or structs, and return a single value, which can also be an array or struct. UDFs written in JavaScript can include external resources, such as encryption or other libraries.

Proprietary + Confidential

BigQuery allows collaborators to save and share queries between team members



Google Cloud

BigQuery allows collaborators to save and share queries between team members.

This feature can be especially useful in data exploration exercises or as a means of coming up to speed on a new dataset or query pattern.

Proprietary + Confidential

Agenda

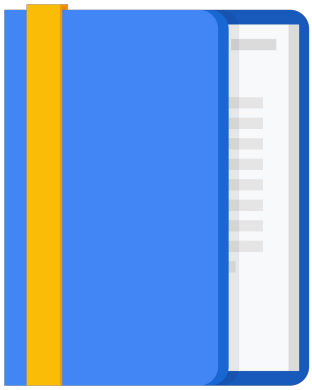
Handling Change and Slowly Changing Dimensions

Querying Data

Lab: Exploring Your Ecommerce Dataset with SQL in BigQuery

Managing Workloads and Concurrency

Analyzing Data



Lab Intro

Exploring Your Ecommerce Dataset with SQL in BigQuery

In this lab, you use a copy of an ecommerce dataset that has millions of Google Analytics records for the Google Merchandise Store loaded into a table in BigQuery. Sample scenarios are provided, from which you look at the data and ways to remove duplicate information. The lab then steps you through further analysis the data.

Proprietary + Confidential

Lab objectives

- Access an ecommerce dataset
- Look at the dataset metadata
- Remove duplicate entries
- Write and execute queries

Proprietary + Confidential

Agenda

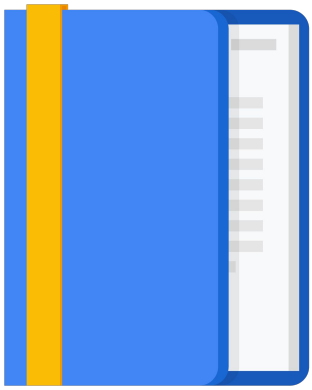
Handling Change and Slowly Changing Dimensions

Querying Data

Lab: Exploring Your Ecommerce Dataset with SQL in BigQuery

Managing Workloads and Concurrency

Analyzing Data



Google Cloud

An effective data warehouse must support a variety of workloads from a large number of concurrent users.

Proprietary + Confidential

Interactive queries count towards query quotas

Google Cloud

To manage resources, BigQuery limits the maximum rate of incoming requests and enforces appropriate quotas on a per-project basis. Specific policies vary depending on resource availability, user profile, service usage history, and other factors.

BigQuery offers two types of query priorities: interactive and batch. By default, BigQuery favors interactive queries, which means that the query is executed as soon as possible. Interactive queries count towards query quotas. Batch queries are queued and executed as soon as idle resources are available, usually within a few minutes.

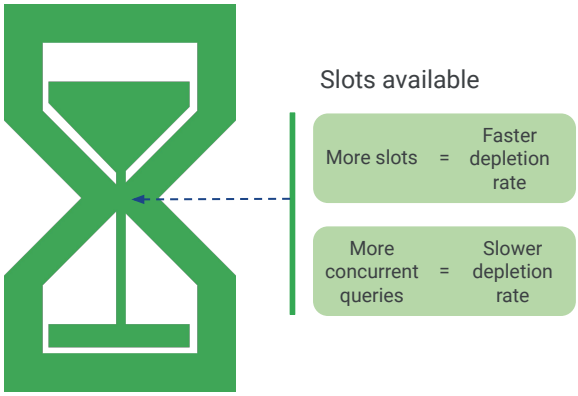
Proprietary + Confidential

Estimating the right BigQuery slots allocation is critical

Guideline: It is recommended to plan for 2,000 BigQuery slots for every 50 medium-complexity queries simultaneously

50 queries = 2,000 Slots

(quota: 100 concurrent queries, but this can be raised)



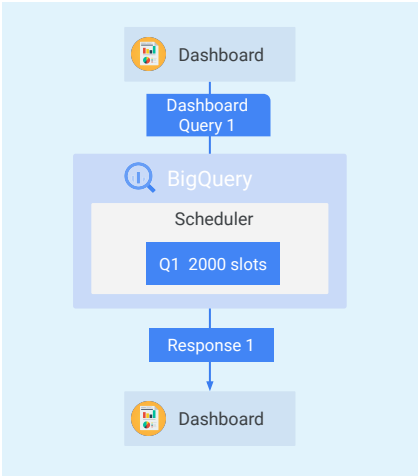
Google Cloud

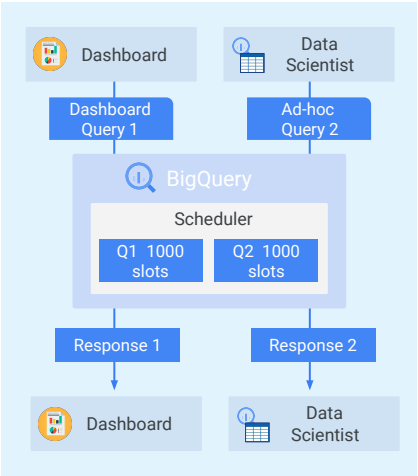
BigQuery doesn't support fine-grained prioritization of interactive or batch queries. It allocates slots to allow timely execution and avoid a pile up of jobs. If there aren't enough slots to run a query in the most efficient manner, BigQuery will allocate fewer slots and the job will take longer.

As a guideline, it's recommended to plan for 2,000 BigQuery slots for every 50 medium-complexity queries simultaneously. The quota is 100 concurrent queries, but this can be raised.

Proprietary + Confidential

BigQuery has a fair scheduler





Google Cloud

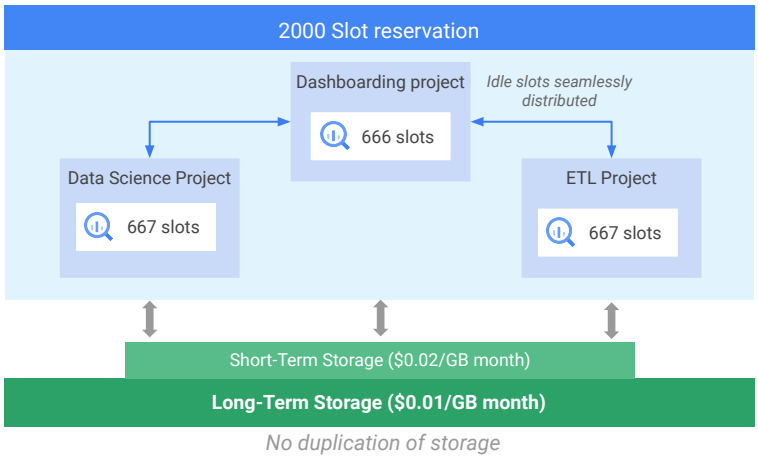
BigQuery allocates slots fairly. If only one query is executing, it has full access to the number of slots available to the project or reservation, by default 2000.

If a second query request arrives, BigQuery will split the slots between the two queries with each getting half the total number of slots available, in this case 1000 each.
This subdividing of compute resources will continue to happen as more queries are executed.

This is a way of saying that it's unlikely one resource-heavy query will overpower the system and steal resources from other running queries.

Proprietary + Confidential

Concurrency is fair across projects, users, and queries, with unused capacity fairly divided among existing tasks



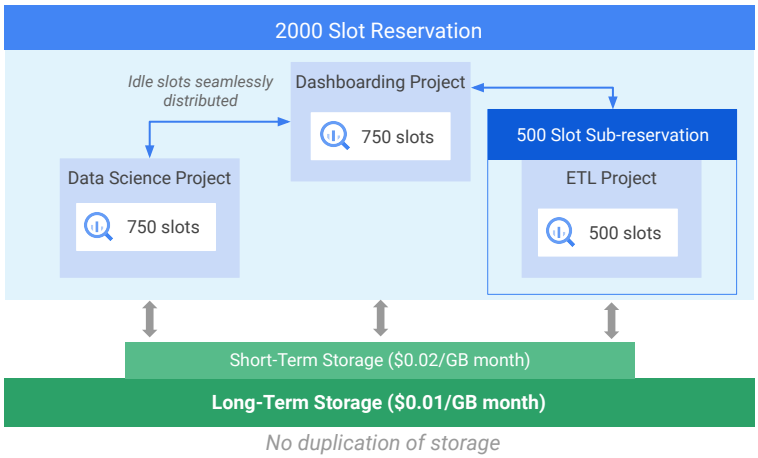
Google Cloud

Flat rate pricing puts a fixed limit on the number of slots. Slot allocation is fair across projects, users, and queries, with unused capacity fairly divided among existing tasks. That is, if you have 2000 slots and two projects, each project can get up to 1000 slots. If one project uses less, the other project will be able to use all of the remainder. If you have two users in each project, each user will be able to get 500 slots. And if each of the two users in each of the two projects runs two queries, they'll each get 500 slots.

This is a way of saying they won't likely degrade performance by adding projects.

Proprietary + Confidential

To prioritize projects, set up a hierarchical reservation



Google Cloud

You can prioritize one project over another by setting up a hierarchical reservation. Let's say you have an ETL project that's somewhat lower priority than your dashboarding project. You can give the ETL project 500 slots as a sub-reservation and the dashboarding project will be in the outer one. If both projects are fully using their reservations, the ETL project can never get more than 500 slots. When one project is lightly used, the other project will be able to take the remaining slots.

Proprietary + Confidential

Agenda

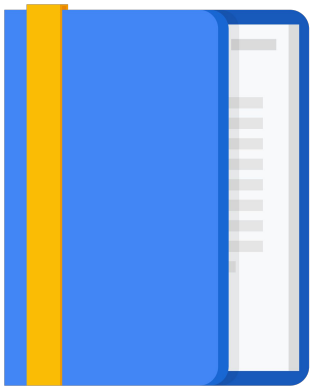
Handling Change and Slowly Changing Dimensions

Querying Data

Lab: Exploring Your Ecommerce Dataset with SQL in BigQuery

Managing Workloads and Concurrency

Analyzing Data



Google Cloud

The primary reason for a data warehouse is to analyze and visualize the data.

Proprietary + Confidential

There are various ways that you can connect to BigQuery and analyze the data

tableau, looker, CHARTIO, Anodot, SAP Analytics Cloud, bime by zendesk, Yellowfin, ZOOMDATA, iCharts, OWOX, atscale, MicroStrategy, Dundas BI, MODE, DOMO, IBM Cognos Analytics, Qlik, re'dash

Data Analysis and Visualization Partners

Google Cloud

To take full advantage of BigQuery as an analytical engine, you should store the data in BigQuery storage. However, specific use cases might benefit from analyzing external sources either by themselves or JOINed with data in BigQuery storage.

Google Data Studio and Looker, as well as many partner tools that are already integrated with BigQuery, can be used to draw analytics from BigQuery and build sophisticated interactive data visualizations.

Proprietary + Confidential

Build custom applications and platforms on top of BigQuery

- Client Libraries: Java, Python, Go, Node, .NET, Ruby, PHP
- JDBC/ODBC Drivers
- BigQuery REST API

Google Cloud

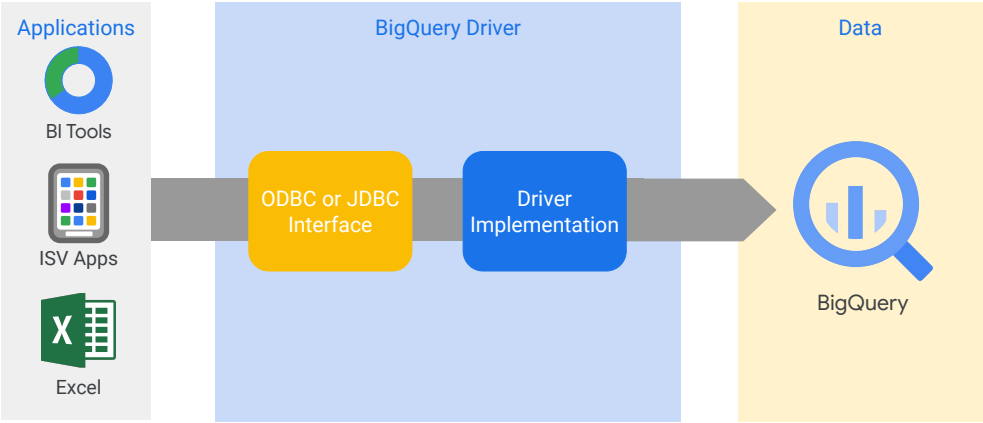
To build custom applications and platforms on top of BigQuery, you can use client libraries, which are available for most common programming languages, or you can use the BigQuery REST API directly.

Note that all of the methods for connecting to BigQuery essentially provide a wrapper around the BigQuery REST API. All connections to the BigQuery API are encrypted by using HTTPS, and enforce permissions by using IAM policies.

Google has partnered with Simba Technologies Inc to provide ODBC and JDBC drivers for the BigQuery Standard SQL.

Proprietary + Confidential

Use third-party connectors to connect to BigQuery from an app that isn't integrated at the API level



Google Cloud

To connect to BigQuery from an application that isn't natively integrated with BigQuery at the API level, you can use the BigQuery JDBC and ODBC drivers. The drivers provide a bridge to interact with BigQuery for legacy applications, independent software vendor applications, or applications that cannot be easily modified, such as Microsoft Excel. Although ODBC and JDBC support interacting with BigQuery using SQL, the drivers aren't as expressive as dealing with the API directly.

Proprietary + Confidential

Agenda

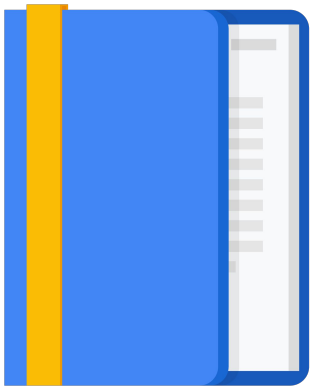
Sizing and Cost Management

Query Optimization

Storage Optimization

Lab: Creating a Data Warehouse Through Joins and Unions

Summary



Google Cloud

Customers always want to know how much it will cost so let's look at how to determine and control the cost of both storage and analytics.

Proprietary + Confidential

Once your data is loaded into BigQuery, you're charged for storing it

Active storage pricing

Storage pricing is prorated per MB, per second. For example, if you store:

- 100 MB for half a month, you pay $0.001 (a tenth of a cent)
- 500 GB for half a month, you pay $5
- 1 TB for a full month, you pay $20

Long-term storage pricing

- Partition or non-partitioned table not edited 90+ consecutive days
- Pricing drops ≈ 50%
- No degradation (performance, durability, availability, functionality)
- Applies to BigQuery storage only

Google Cloud

Storage pricing is based on the amount of data stored in tables when it's uncompressed. The size of the data is calculated based on the data types of the individual columns. Active storage pricing is prorated per megabyte, per second.

If a partition or non-partitioned table isn't edited for 90 consecutive days it's considered long-term storage. The price of storage for that table automatically drops by approximately 50 percent. There's no degradation of performance, durability, availability, or any other functionality.

Proprietary + Confidential

Different actions reset the timer

Activities that reset the timer

- Loading, copying, or streaming data into a partition or non-partitioned table
- Writing query results to a partition or non-partitioned table
- Using Data Definition Language

Activities that don't reset the timer

- Querying a table
- Creating a view that queries a table
- Exporting data from a table
- Copying a table to another destination table
- Patching or updating a table resource

Google Cloud

If the table is edited, the price reverts back to the regular storage pricing, and the 90-day timer starts counting from zero. This is another reason for considering using partitioned tables. Even if it doesn't improve query performance you can save money on storage.

Any actions on a table that don't change the data or the structure of the table don't affect the timer.

Proprietary + Confidential

Which BigQuery pricing model to pick?

On-demand pricing	Flat-rate pricing
• $5/TB of data processed • Quota limit of 2,000 slots • Slots shared amongst all on demand users • 1st TB of data processed is free each month	• Fixed rate pricing is $10k* per 500 slots per month • Slots are dedicated 24/7 • Starting at 500 slots • Unlimited use of slots
Good for **exploratory work** and discovery	Good for **multiple large workloads** and **consistent monthly billing**

Google Cloud

Analytics engine cost is separate from storage costs. The default pricing is “on-demand”, but customers who prefer a consistent bill amount may opt for flat-rate pricing. Capacity is sold in increments of 500 slots with a current minimum of 500 slots. Slots are available in the following commitment plans,

- Flex slots: You commit to an initial 60 seconds.
- Monthly: You commit to an initial 30 days.
- Annual: You commit to 365 days.

Google offers a 25% discount for customers choosing a term length of a least 1-year. Use the Google Cloud Pricing Calculator to find the exact price for the number of slots you want in a region.

Let’s compare the on-demand and flat-rate options in a little more detail.

Proprietary + Confidential

For queries, on-demand pricing is flexible and efficient

- On-demand pricing is based solely on usage
- It doesn't matter where the data is stored
- You can set user-level and project-level custom cost controls
- You can set the maximum bytes billed by query

Google Cloud

Under on-demand pricing, BigQuery charges for queries by using one metric: the number of bytes processed, also referred to as bytes read.

You're charged for the number of bytes processed whether the data is stored in BigQuery or in an external data source such as Cloud Storage, Google Drive, or Cloud Bigtable.

To help control costs you can set user-level and project-level custom quotas, and

You can set the maximum bytes billed by query. If the query will read bytes beyond the limit, the query fails with a "Query exceeded limit for bytes billed" error without incurring any charges.

Proprietary + Confidential

There are a number of things to note regarding query charges

- Charged according to the total data processed in the columns selected and type of data.
- No charge for errors or results from the cache.
- Rounded to nearest MB, min 10 MB.
- Cancelled queries may incur charges.
- Data processed in columns selected supersedes LIMIT set.
- Use partitioning and clustering to reduce data processed.
- On-demand pricing = analysis pricing.

Google Cloud

- BigQuery uses a columnar data structure. You're charged according to the total data processed in the columns you select, and the total data per column is calculated based on the types of data in the column.
- You aren't charged for queries that return an error, or for queries that retrieve results from the cache. Using results from cache is on by default.
- Charges are rounded to the nearest MB, with a minimum 10 MB data processed per table referenced by the query, and with a minimum 10 MB data processed per query.
- Cancelling a running query job may incur charges up to the full cost for the query were it allowed to run to completion.
- When you run a query, you're charged according to the data processed in the columns you select, even if you set an explicit LIMIT on the results. That’s because LIMIT is processed AFTER the query completes.
- Partitioning and clustering tables can help reduce the amount of data processed by queries. As a best practice, use partitioning and clustering whenever possible.
- On-demand query pricing is referred to as analysis pricing on the Google Cloud SKUs page. This is useful if you’re looking at billing reports.

Proprietary + Confidential

Flat-rate pricing offers predictable and consistent month-to-month costs

- Applies to query costs, including DML and DDL statements
- Doesn't apply to storage costs
- Purchased as a regional resource
- Available in monthly and annual commitments
- Customers can raise per-project concurrency quotas by contacting Google Cloud Support

Google Cloud

Let's have a look at flat-rate pricing.

- Flat-rate pricing limits the number of slots used at any time. Slot usage includes resources required for both DML and DDL. If your queries exceed your flat-rate capacity, your queries are queued until your flat-rate resources become available.
- As with fixed-rate pricing, storage costs are calculated independently.
- The flat-rate cost depends on both the number of slots and the region in which the slots are purchased.
- By default, flat-rate pricing is calculated based on a monthly commitment, but you can get a discounted price with an annual commitment. BigQuery limits the rate of incoming requests on a per-project basis. You can contact Google Cloud support to increase quotas, however, additional charges may accrue.

Proprietary + Confidential

Considerations when enrolling for flat-rate pricing

- Flex slots are a special commitment type and a subject to capacity availability.
- Monthly commitments cannot be canceled or downgraded for 30 calendar days from the purchase confirmation date.
- Annual commitments cannot be canceled or downgraded for one calendar year.
- To purchase additional BigQuery slots, you must enter into a new commitment.
- Flat-rate pricing is purchased for a specific BigQuery location.
- Flat-rate and on-demand pricing can be used together.
- To discontinue a flat-rate pricing plan, you must cancel or downgrade your commitment.

Google Cloud

There are a number of considerations for flat-rate pricing.

- Flex slots are a special commitment type.
  - Commitment duration is only 60 seconds.
  - You can cancel Flex slots any time thereafter.
  - You are charged only for the seconds your commitment was deployed.
  - Flex slots are subject to capacity availability. When you attempt to purchase Flex Slots, success of this purchase is not guaranteed. However, once your commitment purchase is successful, your capacity is guaranteed until you cancel it.

- Monthly commitments cannot be canceled for 30 days after your commitment is active. After the first 30 calendar days, you can cancel or downgrade at any time. If you cancel or downgrade, charges are prorated per-second at the monthly rate. For example:
  - You cannot cancel on day 29,
  - If you cancel during the first second of day 31, you're charged for 30 days and 1 second, and
  - If you cancel at the midpoint of the third month, you're charged 50% of your monthly rate for that month.

- Prior to the anniversary of your commitment date, you can choose to renew for another year, or convert it to a monthly or flex commitment. If you move to the monthly rate, you can cancel any time, and you're charged per-second at the

- monthly rate. For example:
    - If you renew for another year after your annual commitment date, you enter into a new annual commitment, and you continue to be charged the yearly commitment rate.
    - Also, if you don't renew for another year after your annual commitment date, you can cancel at any time, and charges are prorated per-second at the monthly rate.

- If you determine you need more BigQuery slots, you can purchase additional increments of 500. However, doing so will create a new commitment.

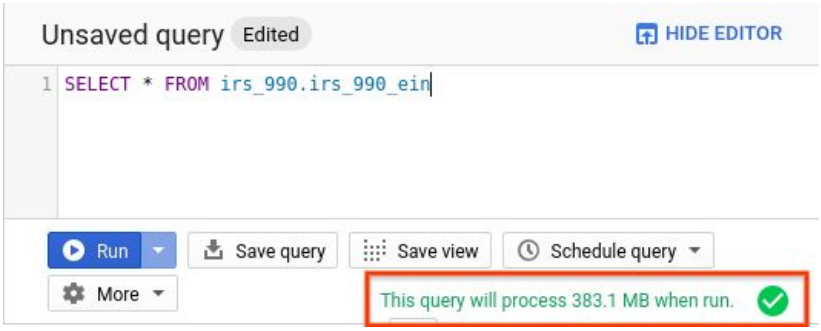
- When you purchase a flat-rate plan, you specify the allocation of slots by location. To use slots in multiple locations, you must purchase slots in each location.

- A project can use either flat-rate or on-demand pricing. If you have multiple projects in a given location, you can choose which projects use flat-rate pricing and which projects use on-demand pricing.

- Lastly, to discontinue a flat-rate pricing plan, you must cancel or downgrade your commitment, but only AFTER the initial commitment period (30 days or 1 year).

Proprietary + Confidential

Estimating query costs: Determine the query size through the Google Cloud console



Google Cloud

On-demand queries are charged based on the number of bytes read. If your query processes a small amount of data, you might need to convert the bytes that are processed from kilobytes to megabytes. A megabyte is the smallest measure that is used by the Google Cloud Pricing Calculator.

When you enter a query in the Google Cloud console, the query validator verifies the query syntax and provides an estimate of the number of bytes read. You can use this estimate to calculate query cost in the Pricing Calculator.

Proprietary + Confidential

Estimating query costs: Determine the query size through the command-line

```
bq --location=[LOCATION] query --use_legacy_sql=false --dry_run 'SELECT
[FIELD1], [FIELD2], [FIELD3] FROM `[PROJECT].[DATASET].[TABLE]` LIMIT
1000'
```

```
Query successfully validated. Assuming the tables are not modified,
running this query will process 10918 bytes of data.
```

Google Cloud

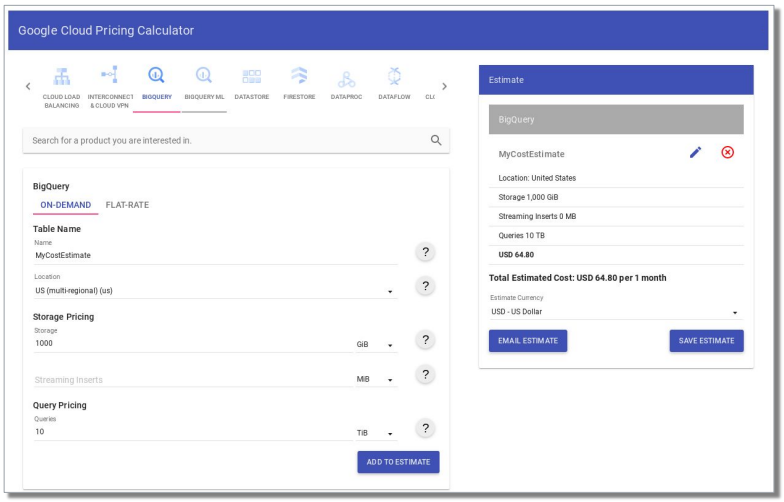
When you run a query in the CLI, you can use the `--dry_run` flag to estimate the number of bytes read. You can use this estimate to calculate query cost in the Pricing Calculator.

When you run the command with dry_run set, the query is not executed, and the response contains the estimated bytes.

You can also perform dry-run estimates by the API or using language libraries for Go, Python, and so on.

Proprietary + Confidential

Estimate on-demand costs



Google Cloud

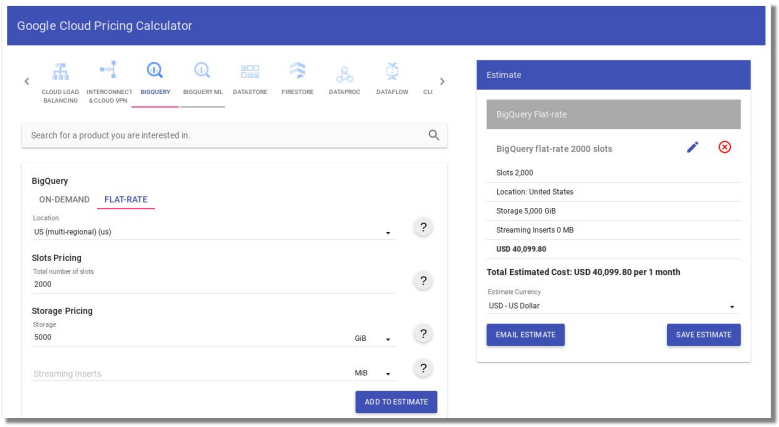
The Google Cloud Pricing Calculator can be used to estimate the on-demand cost of queries or storage, or both.

To estimate on-demand **query** costs, enter the number of bytes that are processed by the query as megabyte, gigabyte, terabyte, or petabyte. If your query processes less than 1 terabyte, the estimate is $0 because BigQuery provides 1 terabyte of on-demand query processing free per month.

To estimate on-demand **storage** costs, enter the number of bytes that are stored as megabyte, gigabyte, terabyte, or petabyte. BigQuery provides 10 gigabytes of storage free per month.

Proprietary + Confidential

Estimate flat-rate costs



https://cloud.google.com/products/calculator/

Google Cloud

If flat-rate pricing is applied to a billing account, you can click the Flat-Rate tab, choose the location and number of slots, then add storage costs to the estimate.

A BigQuery Slot is a proprietary measure of capacity. You can choose a number of slots between 500 and 200000. As discussed, a slot is a combination of CPU, memory, and networking resources. It also includes a number of supporting technologies and sub-services.

The Google Cloud Pricing Calculator can be found at https://cloud.google.com/products/calculator.

Proprietary + Confidential

Control costs (1/5)

Avoid `SELECT *`

Query only the columns that you need.

Using `SELECT *` is the most expensive way to query data.

Sample data using preview options

Use table preview options to view data for free and without affecting quotas.

Google Cloud

Regardless of whether you choose fixed-rate or on-demand pricing, it's important to control costs. Here are a few practices that will help keep costs under control.

- You should query only the columns that you need. Using SELECT * is the most expensive way to query data. When you use SELECT *, BigQuery does a full scan of every column in the table. If you're experimenting with data or exploring data, use one of the data preview options instead of SELECT *. Applying a LIMIT clause to a SELECT * query doesn't affect the amount of data read.

  You're billed for reading all bytes in the entire table, and the query counts against the free tier quota. Instead, query only the columns you need. For example, use SELECT * EXCEPT to exclude one or more columns from the results. If you do require queries against every column in a table, but only against a subset of data, consider materializing results in a destination table and querying that table instead. Also, consider partitioning your tables by date and querying the relevant partition; for example, WHERE _PARTITIONDATE="2019-01-01" only scans the January 1, 2019 partition.

- Don't run queries to explore or preview table data. If you're experimenting with or exploring your data, you can use table preview options to view data for free and without affecting quotas.

Proprietary + Confidential

Control costs (2/5)

Price your queries before running them

Before running queries, preview them to estimate costs.

Queries are billed according to the number of bytes read.

Limit query costs by restricting the number of bytes billed

Use the maximum bytes billed setting to limit query costs.

```
Error: Query exceeded limit for
bytes billed: 1000000. 10485760 or
higher required.
```

Google Cloud

- To estimate costs before running a query use:
  - The query validator in the Google Cloud console or the classic web UI,
  - The --dry_run flag in the CLI,
  - The dryRun parameter when submitting a query job using the API, and
  - The Google Cloud Pricing Calculator.

- You can limit the number of bytes billed for a query using the maximum bytes billed setting. When you set maximum bytes billed, if the query will read bytes beyond the limit, the query fails without incurring a charge. If a query fails because of the maximum bytes billed setting, an error is returned. There are different ways to set the maximum bytes billed.
  - In the classic BigQuery web UI, enter an integer in the Maximum Bytes Billed field in the query options. Currently, the Google Cloud console doesn't support the Maximum Bytes Billed option.
  - In the CLI, use the bq query command with the --maximum_bytes_billed flag.
  - Or, in the API, set the maximumBytesBilled property in the job configuration.

Proprietary + Confidential

Control costs (3/5)

LIMIT doesn't affect cost

Do not use a `LIMIT` clause as a method of cost control.

Applying a `LIMIT` clause to a query doesn't affect the amount of data that's read or how much the query costs.

View costs using a dashboard and query your audit logs

Create a dashboard to view your billing data so you can make adjustments to your BigQuery usage.

Google Cloud

- Don't use a LIMIT clause as a method of cost control. Applying a LIMIT clause to a query doesn't affect the amount of data that's read. It merely limits the results set output. You're billed for reading all bytes in the entire table as indicated by the query.
- You can export billing data to BigQuery and visualize it in a tool such as Google Data Studio. For a tutorial on creating a billing dashboard, see Visualize Google Cloud billing using BigQuery and Google Data Studio. You can also stream audit logs to BigQuery and analyze the logs for usage patterns such as query costs by user.

Proprietary + Confidential

Control costs (4/5)

Partition data by date

If possible, partition your BigQuery tables by date.

Materialize query results in stages

If possible, materialize your query results in stages.

Google Cloud

- If possible, partition BigQuery tables by date. Partitioning tables allows you to query relevant subsets of data, which improves performance and reduces costs. When you query partitioned tables, use the _PARTITIONTIME pseudo column to filter for a date or a range of dates. The query processes data only in the partitions that are specified by the date or range.
- If you create a large, multi-stage query, each time you run it, BigQuery reads all the data that's required by the query. You're billed for all the data that's read each time the query is run. Instead, break the query into stages, where each stage materializes the query results by writing them to a destination table. Querying the smaller destination table reduces the amount of data that's read and lowers costs. The cost of storing the materialized results is much less than the cost of processing large amounts of data.

Proprietary + Confidential

Control costs (5/5)

Consider the cost of large result sets

Use the default table expiration time to remove the data when it's no longer needed.

Use streaming inserts with caution

There's no charge for loading data into BigQuery.

There is a charge, however, for streaming data into BigQuery.

Google Cloud

- Keeping large result sets in BigQuery storage has a cost. If permanent access to the results isn't needed, use the default table expiration to automatically delete the data.
- There's no charge for batch loading data into BigQuery. There is a charge, however, for streaming data into BigQuery. Unless data must be immediately available, load it in batches rather than streaming it.

Proprietary + Confidential

There are many limits and quotas that apply to BigQuery

- Load jobs limits
- Export jobs limits
- Query jobs limits
- Copy jobs limits
- Streaming insert limits
- Dataset limits
- Table limits
- View limits
- DML limits
- UDF limits

https://cloud.google.com/bigquery/quotas

Google Cloud

There are many limits and quotas that apply to BigQuery. You can find all of the quotas/limit information at **https://cloud.google.com/bigquery/quotas**

Some limits, such as the maximum number of table operations per day, can be very important when deciding how to structure data and how you plan to load and update it over time. You're limited to 1,000 operations per table per day whether the operation appends data to a table, overwrites a table, or uses a DML INSERT statement to write data to a table.

Proprietary + Confidential

Agenda

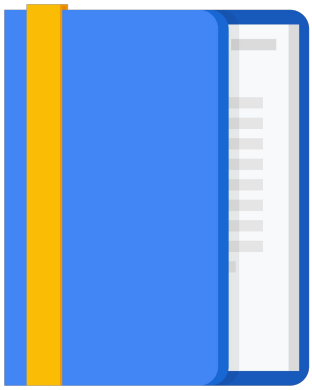
Sizing and Cost Management

Query Optimization

Storage Optimization

Lab: Creating a Data Warehouse Through Joins and Unions

Summary



Google Cloud

In general, queries that do less work perform better. When evaluating query performance in BigQuery, the amount of work required depends on a number of factors:

- Input data and data sources (I/O): How many bytes does your query read?
- Communication between nodes (shuffling): How many bytes does your query pass to the next stage? How many bytes does your query pass to each slot?
- Computation: How much CPU work does your query require?
- Outputs (materialization): How many bytes does your query write?
- Query anti-patterns: Are your queries following SQL best practices?

In this lesson, we'll discuss each of these individually.

Proprietary + Confidential

Manage input data and data sources (1/4)

Control projection - Avoid `SELECT *`

Query only the columns that you need.

Using `SELECT *` is the most expensive way to query data.

Use `SELECT * EXCEPT` to exclude columns.

Use one of the data preview options instead of `SELECT *` to explore data.

Prune partitioned queries

When querying a partitioned table, use the `_PARTITIONTIME` pseudo column to filter the partitions.

```
WHERE _PARTITIONTIME
BETWEEN TIMESTAMP("20190101")
    AND TIMESTAMP("20190131")
```

Google Cloud

- Projection refers to the columns that are read by your query. Projecting excess columns incurs additional (wasted) I/O and materialization (writing results). Querying a subset of data or using SELECT * EXCEPT can greatly reduce the amount of data that is read by a query. In addition to the cost savings, performance is improved by reducing the amount of data I/O and the amount of materialization that's required for the query results. If you're experimenting with data or exploring data, use one of the data preview options instead of SELECT *. Previews don't cost anything and don't count against quotas or limits.

- Filtering the data using _PARTITIONTIME allows you to specify a date or range of dates. For example, this WHERE clause uses the _PARTITIONTIME pseudo column to specify partitions between January 1, 2019 and January 31, 2019. The query processes data only in the partitions that are indicated by the date range, reducing the amount of input data. Filtering your partitions improves query computation performance and reduces costs.

Proprietary + Confidential

Manage input data and data sources (2/4)

Denormalize data when needed

BigQuery performs best when data is denormalized.

Increases in storage costs are worth the performance gains from denormalizing data.

Use nested and repeated fields to maintain relationships.

Google Cloud

Rather than preserving a relational schema, such as a star or snowflake schema, consider denormalizing your data to take advantage of nested and repeated fields.

Joins require data coordination (communication bandwidth). Denormalization localizes the data to individual slots so execution can be done in parallel.

Completely flattening data increases shuffling which impacts query performance so take advantage of nested and repeated fields to denormalize.

Proprietary + Confidential

Use nested and repeated fields

- Nesting data (STRUCT)
- Repeated data (ARRAY)
- Nested and repeated data (ARRAY of STRUCTs)

Google Cloud

There's a trade-off between normalization and denormalization. As discussed earlier, not every table is a good candidate for denormalization. In some circumstances, denormalizing your data and using nested and repeated fields may not improve performance. Avoid denormalization if you have a star schema with frequently changing dimensions. BigQuery complements an Online Transaction Processing (OLTP) system with row-level mutation, but can't replace it.

BigQuery doesn't require a completely flat denormalization. You can use nested and repeated fields to maintain relationships. Nesting data allows you to represent foreign entities inline.

- Nested data is represented as a STRUCT type in standard SQL. A STRUCT is a container of ordered fields, each of which has a data type and name. Querying nested data uses "dot" syntax to reference leaf fields, which is similar to the syntax using a join.
- Repeated data is represented as an ARRAY. You can use an ARRAY function in standard SQL when you query the repeated data. Creating a field of type RECORD with the mode set to REPEATED allows you to preserve a one-to-many relationship inline, so long as the relationship isn't high cardinality. With repeated data, shuffling isn't necessary.
- Nesting and repetition complement each other. For example, in a table of transaction records, you could include an ARRAY of line item STRUCTs.

Proprietary + Confidential

Manage input data and data sources (3/4)

Use external data sources appropriately

If query performance is a top priority, do not use an external data source. Use an external data source for these use cases:

- Performing ETL operations when loading data
- Frequently changing data
- Periodic loads such as recurring ingestion of data from Cloud Bigtable

Google Cloud

Querying tables in BigQuery managed storage is typically much faster than querying external tables in Cloud Storage, Google Drive, or Cloud Bigtable.

Use an external data source when performing ETL operations when loading data, for frequently changing data, or for periodic loads, such as recurring ingestion of data from Cloud Bigtable.

Proprietary + Confidential

Manage input data and data sources (4/4)

Avoid excessive wildcard tables (1/2)

When querying wildcard tables, use the most granular prefix possible.

Use wildcards to query multiple tables by using concise SQL statements.

```
FROM bigquery-public-data.noaa_gsod.gsod194*
```

```
FROM bigquery-public-data.noaa_gsod.gsod.*
```

Google Cloud

Wildcard tables create a union of tables that match the wildcard expression. Wildcard tables are useful if a dataset contains multiple, similarly named tables with compatible schemas called sharded tables.

If your data has fewer than 4000 days (about 11 years) it's generally better to use time-partitioned tables instead of sharded tables.

When you query a wildcard table, specify a wildcard after the common table prefix. For example, FROM bigquery-public-data.noaa_gsod.gsod194* queries all tables from the 1940s.

More granular prefixes perform better than shorter prefixes. For example, the first FROM statement performs better than the second one because fewer tables match the wildcard, and the use of _TABLE_SUFFIX allows you to make even finer grained queries without requiring you to specify each individual table.

Proprietary + Confidential

Manage input data and data sources (4/4)

Avoid excessive wildcard tables (2/2)

When querying wildcard tables, use the most granular prefix possible.

Use wildcards to query multiple tables by using concise SQL statements.

Use the _TABLE_SUFFIX pseudo column to limit wildcard results.

```
SELECT  max FROM  `bigquery-public-data.noaa_gsod.gsod19*`
WHERE _TABLE_SUFFIX BETWEEN '29' AND '40'
```

Google Cloud

Using the _TABLE_SUFFIX pseudo column in the WHERE clause tells BigQuery to only access tables with names that match the wildcard character. In the example above only tables with names gsod1929 to gsod1940 would be read. If the query used WHERE year BETWEEN 1929 and 1940 all gsod tables (1929 to current year) would be accessed.

Proprietary + Confidential

Optimize communication between slots (1/2)

Avoid tables sharded by date

Don't use tables sharded by date (also called date-named tables) in place of time-partitioned tables.

Partitioned tables perform better than date-named tables.

Google Cloud

When you create tables sharded by date, BigQuery must maintain a copy of the schema and metadata for each date-named table. Also, when date-named tables are used, BigQuery might be required to verify permissions for each queried table. This practice also adds to query overhead and impacts query performance.

Proprietary + Confidential

Optimize communication between slots (2/2)

Don't treat `WITH` clauses as prepared statements

`WITH` clauses are used primarily for readability because they aren't materialized.

Reduce data before using a `JOIN`

Reduce the amount of data that is processed before a `JOIN` clause.

Trim the data as early in the query as possible, before the query performs a `JOIN`.

Google Cloud

- Use WITH clauses primarily for readability. Placing all your queries in WITH clauses and then running UNION ALL is a misuse of the WITH clause. If a query appears in more than one WITH clause, it executes in each clause.
- Reduce data early in the processing cycle, so shuffling and other complex operations need only execute on the data that you need.

Proprietary + Confidential

Optimize query computation (1/3)

Avoid repeatedly transforming data via SQL queries

Avoid situations where you repeatedly transform the same data.

Query the destination table without the added transformation overhead.

Avoid JavaScript user-defined functions

Avoid using JavaScript user-defined functions.

Use native UDFs instead.

Google Cloud

- Avoid transforming data in your queries because it increases overhead. Instead, do the transformations during ingestion and store the transformed data in the data warehouse. The rule in OLTP systems is “never store what you can compute”, but in OLAP systems it’s “never compute in queries what you can store in a table”.
- Calling a JavaScript UDF requires the instantiation of a subprocess. Spinning up this process and running the UDF directly impacts query performance. If possible, use a built-in function or a native (SQL) UDF instead.

Proprietary + Confidential

Optimize query computation (2/3)

Use approximate aggregation functions

If your use case supports it, use an approximate aggregation function, they are much faster.

For example, instead of using `COUNT(DISTINCT)`, use `APPROX_COUNT_DISTINCT()`

Order query operations to maximize performance

Use `ORDER BY` only in the outermost query or within window clauses (analytic functions). Push complex operations to the end of the query.

Google Cloud

- Using approximate aggregate functions yield faster results at lower cost. Approximate functions can be useful for use-cases like “what products are trending” but aren’t suitable for corporate financial statements. You can also use HyperLogLog++ function for approximations. Google HyperLogLog for more information.
- Always filter before sorting to reduce the amount of data to sort. An ORDER BY clause should appear only in the outermost query unless your query includes windowing functions such as RANK that assign values in a particular order. Placing an ORDER BY clause in the middle of a query greatly impacts performance and provides no value because data will be sorted again at the end. Another technique for ordering your query is to push complex operations, such as regular expressions and mathematical functions to the end of the query. Again, this technique allows the data to be pruned as much as possible before the complex operations are performed.

Proprietary + Confidential

Optimize query computation (3/3)

Optimize your join patterns

For queries that join data from multiple tables, optimize your join patterns.

Use a large table as the left side and a small one on the right side.

Google Cloud

The order in which tables are joined impacts performance. The standard SQL query optimizer can determine which table should be on which side of the join, but it's still recommended to order your joined tables appropriately.

The best practice is to JOIN the largest table to the smallest table first, and then JOIN additional tables by decreasing size.

When you have a large table as the left side of the JOIN and a small one on the right side of the JOIN, BigQuery performs a broadcast join. A broadcast join sends all the data in the smaller table to each slot that processes the larger table. It's advisable to perform the broadcast join first.

Proprietary + Confidential

Manage query outputs (1/2)

Avoid repeated joins and subqueries

If you're repeatedly joining the same tables, revisit your schema and use nested repeated data instead.

Consider materializing large result sets to a destination table

BigQuery limits cached results to approximately 10 GB compressed.

Overcome the limitation on cached result size by:

- Using filters
- Using a `LIMIT` clause
- Write the output data to a table

Google Cloud

- Avoid repeatedly joining the same tables and using the same subqueries. If you're repeatedly joining the same tables, revisit your schema. Instead, use nested repeated data to represent the relationships. Nested repeated data saves you the performance impact of the communication bandwidth that's required by a join. It also saves you the I/O costs that are incurred by repeatedly reading and writing the same data. Similarly, repeating the same subqueries impacts performance through repetitive query processing. If you're using the same subqueries in multiple queries, consider materializing the subquery results in a table. Then consume the materialized data in your queries. Materializing your subquery results improves performance and reduces the overall amount of data that is read and written by BigQuery. The small cost of storing the materialized data outweighs the performance impact of repeated I/O and query processing.

- BigQuery limits cached results to approximately 10 gigabytes compressed. Queries that return larger results overtake this limit and frequently result in the error: Response too large. Overcome the limitation on cached result size by using filters to limit the size of the result set, use a LIMIT clause to reduce the result set, especially if you using an ORDER BY clause, and consider writing the output data to a destination table. The "response too large" error often occurs when you select a large number of fields from a table with a considerable amount of data. Issues writing cached results can also occur in ETL-style queries that normalize data without reduction or aggregation. Be aware that writing very large result sets to destination tables impacts query performance (I/O). In addition, you'll incur a small cost for storing the

- destination table. You can automatically delete a large destination table by using the dataset default table expiration.

Proprietary + Confidential

Manage query outputs (2/2)

Use a LIMIT clause with large sorts

If you're sorting a very large number of values, use a LIMIT clause.

```
SELECT
  title
FROM
  `cloud-training-demos.wikipedia_benchmark.Wiki1B`
ORDER BY
  title
```

```
SELECT
  title
FROM
  `cloud-training-demos.wikipedia_benchmark.Wiki1B`
ORDER BY
  title DESC
LIMIT
  1000
```

Google Cloud

While a LIMIT clause doesn't reduce the number of rows that are read from source tables, it CAN be used to reduce the amount of sorting that must be done.

For example, this first query orders a very large results set and throws a Resources exceeded error. The query sorts by the title column in the Wiki1B table. The title column contains millions of values.

The second query removes the error because once the first 1000 rows are found the process ends.

Proprietary + Confidential

Avoid SQL anti-patterns (1/3)

Data skew

To avoid performance issues that result from data skew:

- Use an approx aggregate function such as `APPROX_TOP_COUNT` to determine if the data is skewed
- Filter your data as early as possible

To avoid performance issues that are associated with unbalanced joins:

- Pre-filter rows from the table with the unbalanced key
- If possible, split the query into two queries

Google Cloud

Partition skew, sometimes called data skew, is when data is partitioned into very unequally sized partitions. This creates an imbalance in the amount of data sent between slots. You can't share partitions between slots, so if one partition is especially large, it can slow down, or even crash the slot that processes the oversized partition.

If your query processes keys that are heavily skewed to a few values, filter your data as early as possible or use approximate aggregations.

Partitions become large when your partition key has a value that occurs more often than any other value. For example, grouping by a user_id field where there are many entries for guest or NULL.

When a slot's resources are overwhelmed, a **Resources exceeded** error results. Reaching the shuffle limit for a slot (2 TB in-memory compressed) also causes the shuffle to write to disk and further impacts performance. Customers with flat-rate pricing can increase the number of allocated slots.

If the query explain plan shows a significant difference between average and maximum compute times, the data is probably skewed.

Data skew can also appear when you use JOIN clauses. Because BigQuery shuffles data on each side of the join, all data with the same join key goes to the same shard. This shuffling can overload the slot.

Proprietary + Confidential

Avoid SQL anti-patterns (2/3)

Self-joins

Avoid self-joins.

Use a window function instead.

Cross joins (Cartesian product)

Avoid joins that generate more outputs than inputs.

When a `CROSS JOIN` is required:

- Use a `GROUP BY` clause to pre-aggregate the data.
- Use a window function.

Google Cloud

- Typically, self-joins are used to compute row-dependent relationships. The result of using a self-join is that it potentially doubles the number of output rows. This increase in output data can cause poor performance. Instead of using a self-join, use a window (analytic) function to reduce the number of additional bytes that are generated by the query.
- CROSS JOIN is where every row of the first table is joined to every row in the second table. If the first table had 1000 rows and the second table had 1000 rows, the result would be 1 million rows. If CROSS JOIN is required, filter and pre-aggregate each table using a subquery before joining. If possible, consider rewriting the query to use Window functions instead of CROSS JOIN.

Proprietary + Confidential

Avoid SQL anti-patterns (3/3)

DML statements that update or insert single rows

Avoid point-specific DML statements (updating or inserting 1 row at a time).

Batch your updates and inserts.

```
UPDATE
  dataset.t t
SET
  my_column = u.my_column
FROM
  dataset.u u
WHERE
  t.my_key = u.my_key
```

Google Cloud

Using point-specific DML statements is an attempt to treat BigQuery like an Online Transaction Processing (OLTP) system. BigQuery focuses on Online Analytical Processing (OLAP) by using table scans and not point lookups. If you need OLTP-like behavior (single-row updates or inserts), consider a database designed to support OLTP use cases such as Cloud SQL.

BigQuery DML statements are intended for bulk updates. UPDATE and DELETE DML statements in BigQuery are oriented towards periodic rewrites of your data, not single row mutations. The INSERT DML statement is intended to be used sparingly. Inserts consume the same modification quotas as load jobs. If your use case involves frequent single row inserts, consider streaming your data instead.

If batching your UPDATE statements yields many tuples in very long queries, you might approach the query length limit of 256 KB. To work around the query length limit, consider whether your updates can be handled based on a logical criteria instead of a series of direct tuple replacements.

For example, you could load your set of replacement records into another table, then write the DML statement to update all values in the original table if the non-updated columns match. If the original data is in table t and the updates are staged in table u, the query would look as shown in the example.

Proprietary + Confidential

Agenda

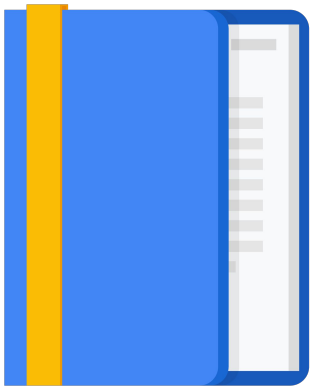
Sizing and Cost Management

Query Optimization

Storage Optimization

Lab: Creating a Data Warehouse Through Joins and Unions

Summary



Google Cloud

The other half of the cost equation is data storage, which refers to data that's stored in BigQuery storage not in federated data sources. As with queries, you can optimize performance and cost for storage as well.

Proprietary + Confidential

Use the expiration settings to remove unneeded tables and partitions

Configure the default table expiration for your datasets, configure the expiration time for your tables, and configure the partition expiration for partitioned tables.

```
bq mk --time_partitioning_type=DAY --time_partitioning_expiration=259200
[DATASET].[TABLE]
```

Google Cloud

The less data you store, the lower the cost. One way to reduce the amount of data stored is by setting the default expiration for a dataset or individual expiration for either tables or partitions. When you set this property, BigQuery automatically deletes data that has expired.

You can control expiration using the UI, CLI, or the expiration configuration setting in the API by specifying a default lifetime, in seconds, for the partitions. The command shown here creates a table partitioned on ingestion date where partitions expire after 3 days.

Proprietary + Confidential

Take advantage of long-term storage

Keep your data in BigQuery.

Load data into BigQuery at no cost.
Rather than exporting older data to another storage option,
take advantage of BigQuery's long-term storage pricing.

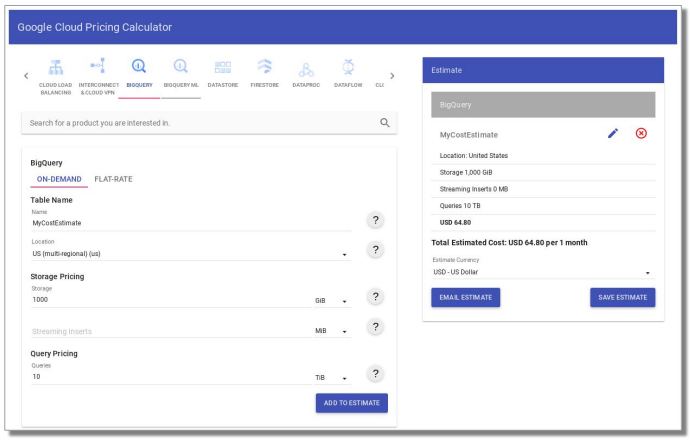
Google Cloud

If you have a table that isn’t changed for 90 consecutive days, the price of storage for that table automatically drops by 50 percent. This is the same cost as Cloud Storage Nearline.

Each partition of a partitioned table is considered separately for long-term storage pricing. If a partition hasn't been modified in the last 90 days, the data in that partition is considered long-term storage and is charged at the discounted price.

Proprietary + Confidential

Estimate storage costs using the Google Cloud Pricing Calculator



Google Cloud

You can use the Google Cloud Pricing Calculator to estimate costs for all services. BigQuery pricing includes storage, query, and streaming data, which is discussed in the Streaming Analytics module. Storage estimates take into account both the amount of storage and the location.

Proprietary + Confidential

Agenda

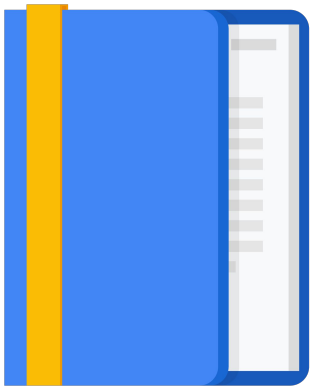
Sizing and Cost Management

Query Optimization

Storage Optimization

Lab: Creating a Data Warehouse Through Joins and Unions

Summary



Lab Intro

Creating a Data Warehouse Through Joins and Unions

This lab focuses on how to create new reporting tables using SQL JOINS and UNIONs.

Proprietary + Confidential

Lab objectives

- Explore new ecommerce data on sentiment analysis
- Join together datasets and create new tables
- Append historical data with unions and table wildcards

Proprietary + Confidential

Agenda

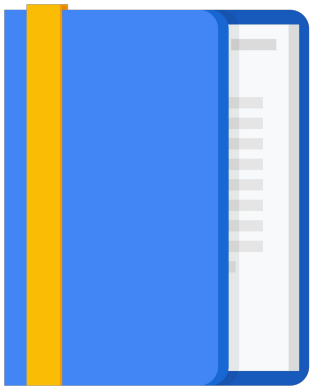
Sizing and Cost Management

Query Optimization

Storage Optimization

Lab: Creating a Data Warehouse Through Joins and Unions

Summary



Proprietary + Confidential

Summary (1/3)

BigQuery organizes data tables into units called datasets.

You don't need to provision resources before using BigQuery.

A BigQuery slot is a combination of CPU, memory, and networking resources.

BigQuery is a fully-managed service and also addresses backup and disaster recovery at the service level.

BigQuery provides predefined roles for controlling access to resources.

Cloud Audit Logs can provide BigQuery-specific log information.

Log exports from Cloud Logging to BigQuery allows for aggregated analysis on log data.

Google Cloud

- In BigQuery, datasets are used to logically segment data and views much like schemas in other databases. Security can be assigned at the dataset level, so a common practice is to create a dataset for tables that are accessible to developers and ETL processes. Another dataset is used for views that data scientists and analysts can access. Separate datasets are also used to store data that would be considered a data mart because they are accessed individually by some users and others have access to multiple data marts.

- BigQuery is serverless and fully managed so there's nothing to configure or manage. This frees up people to do more interesting and valuable work.

- BigQuery uses a microservice architecture in which usage is measured using the concept of a slot. Slots aren't the equivalent of virtual machines or containers – they're simply units of computation capacity.

- Because BigQuery is fully managed, there's no need to do backups or perform other administrative functions. You can restore data from any point in time within the last 7 days and can even recover deleted tables for up to 2 days.

- Rather than have a separate system for security like other databases, BigQuery uses IAM just like all other resources within Google Cloud. The use of predefined or user-defined roles combined with groups simplifies access management.

- Google Cloud automatically creates logs for every service. BigQuery logs include information about both objects and data.

- Logs can be exported to BigQuery where they can be maintained for longer than the usual 30-day retention. Having logs in BigQuery also simplifies access because users can write SQL queries to get results and then visualize them in Data Studio.

Proprietary + Confidential

Summary (2/3)

You can organize datasets to manage workflows.

Consider when to denormalize versus utilizing a star schema and joins based on query and data insert patterns.

BigQuery supports clustering over a partitioned table.

BigQuery supports ELT and ETL approaches to loading data.

BigQuery manages service availability, but you control the duration and timeliness of your datasets.

BigQuery supports standard SQL queries and user-defined functions for queries that exceed the complexity of SQL.

Google Cloud

- Because datasets can have their own security settings they make great building blocks when you're designing workflows.
- BigQuery supports both normalized and denormalized structures so you can design your data warehouse in a way that maximizes performance and usability.
- To facilitate access to data stored in BigQuery, you can partition data on ingestion date, date column, or an integer column. You can also create clusters within the partitions to access data faster.
- You can use a variety of methods and tools to load data into BigQuery. You'll be introduced to several methods later in the course.
- The BigQuery service is fully managed, however you have options for controlling your own datasets. You can get BigQuery to do the work for you if you define expirations on datasets, tables, or partitions.
- It's easy to extract data using standard SQL, but if you need more complex functionality you can create user-defined functions in SQL or javascript.

Proprietary + Confidential

Summary (3/3)

The BigQuery web UI allows interactive dataset querying

BigQuery enforces appropriate quotas on a per-project basis

Storage pricing is based on the amount of data stored in your tables. A table not edited for 90 consecutive days is considered long-term storage

Query sizes can be estimated using the Google Cloud console, classic BigQuery web UI, or command-line before using the Google Cloud Pricing Calculator to estimate on-demand or flat-rate costs

Best practices can be applied to control costs as well as optimize queries and storage

Limits apply to loading, querying, and exporting data, as well as copying tables in BigQuery

Google Cloud

- You can use the web UI from the Google Cloud console as an easy way to write and execute queries in BigQuery. And you can directly visualize query results in Data Studio by simply clicking a button.
- There are some Google-defined quotas on BigQuery, but you can create your own custom quotas on a per-project or per-user basis to control costs.
- You only pay for the BigQuery storage that you use. A 50% discount is provided for data that hasn't been modified for 90 days.
- Before running a query you can get an estimate of the amount of data that will be processed. The BigQuery web UI automatically displays this value but you can also request it using the CLI. This information is useful when analyzing the potential costs for queries.
- There are many ways to control BigQuery costs and I discussed most of them. When you're implementing your solution, you can refer to the recommended practices in the Google Cloud online documentation.
- Quotas (or limits) are set by Google on most operations in order to help avoid billing "surprises". Most quotas can be increased by contacting your friendly Google rep.